ASSIGNMENT COVERSHEET

UTS: ENGINEERING & INFORMATION TECHNOLOGY

SUBJECT NUMBER & NAME	NAME OF STUDENT(s) (PRINT CLEARLY)	STUDENT ID(s)
31256 Image Processing and Pattern Recognition	Anush Harutyunyan Jayden Tran Minseok Lee Qian Zhao Fabiha Tabassum Priti Yaeesh Mahomed *SURNAME* *FIRST NAME*	24834362 14514614 25410085 25777830 14505578 24957692

STUDENT EMAIL	STUDENT CONTACT NUMBER
anush.harutyunyan@student.uts.edu.au fabiha.t.priti@student.uts.edu.au jayden.k.tran@student.uts.edu.au minseok.lee@student.uts.edu.au qian.zhao-5@student.uts.edu.au yaeesh.mahomed@student.uts.edu.au	Anush Harutyunyan : 0466895930 Jayden Tran : 0497377779 Minseok Lee : 0424279948 Qian Zhao : 0493210835 Fabiha Tabassum Priti : 0436343821 Yaeesh Mahomed : 0424870356

NAME OF TUTOR	TUTORIAL GROUP	DUE DATE
Stuart Perry	01, 41	03/11/2025

ASSESSMENT ITEM NUMBER & TITLE
Assignment 2: Project Implementation

☑ I acknowledge that if AI or another nonrecoverable source was used to generate materials for background research and self-study in producing this assignment, I have checked and verified the accuracy and integrity of the information used.
☑ I confirm that I have read, understood and followed the guidelines for assignment submission and presentation on page 2 of this cover sheet.
☑ I confirm that I have read, understood and followed the advice in the Subject Outline about assessment requirements.
☑ I understand that if this assignment is submitted after the due date, it may incur a penalty for lateness unless I have previously had an extension of time approved and have attached the written confirmation of this extension.

Declaration of originality: The work contained in this assignment, other than that specifically attributed to another source, is that of the author(s) and has not been previously submitted for assessment. I have rewritten any material provided by AI or other nonrecoverable sources and where appropriate acknowledged their contribution. I understand that, should this declaration be found to be false, disciplinary action could be taken and penalties imposed in accordance with University policy and rules. In the statement below, I have indicated the extent to which I have collaborated with others, whom I have named.

No content generated by AI technologies or other sources has been presented as my own work and I have rewritten any text provided by AI or other sources in my own words.

Statement of collaboration:

Signature of student(s) AH
Signature of student(s) JT
Signature of student(s) ML
Signature of student(s) QZ
Signature of student(s) FTP
Signature of student(s) YM

ASSIGNMENT RECEIPT

SUBJECT NUMBER & NAME	NAME OF TUTOR

SIGNATURE OF TUTOR	RECEIVED DATE

STYLE GUIDE for ASSIGNMENT SUBMISSION

Before submitting an assignment, you should refer to the policies and guidelines set out in the following:

- FEIT Student Guide
- UTS Library - referencing
- HELPS - English and academic literacy support
- UTS GSU - coursework assessment policy and procedures

Unless your Subject Coordinator has indicated otherwise in the Subject Outline, you must follow the instructions below for submission of assignments in the Faculty of Engineering and Information Technology.

Writing style

It is usually best to write your initial draft in the default settings of your software without formatting. Use the following guides in your writing.

Purpose and audience: use the correct genre and language style expected for the particular task.

Language: use 'plain English' for all technical writing. More information about this language style can be found at www.plainenglish.co.uk/free-guides.html.

Use spelling and grammar software tools to check your writing. Edit your document.

Standards: always use:

- Australian spelling standards (Macquarie Dictionary)
- SI (International System of Units) units of measurement
- ISO (International Organisation for Standardisation) for writing dates and times for international documents. For example **yyyy-mm-dd** or **hh-mm-ss**. However, for most applications it is more helpful to present the date in full as **26 August 2016**.

Graphics and tables should:

- be numbered
- have an appropriate heading and/or caption
- be fully labelled
- be correctly referenced.

Presentation

Unless otherwise instructed, all assignment submissions should be **word processed** using spell-check and grammar-check software. Work should be well **edited** before submission. Use the following default settings:

Page setup: set margins at no less than 20mm all around.
Paper: print on A4 bond, double-spaced and preferably double-sided, left justified.

Font: use the software default style to provide consistency. The recommended style includes:

- 10-12 pt font
- consistent formatting with a limited number of fonts
- lines no more than 60 characters (use wider margins or columns if you need to make lines shorter)

Header should include:

- your name and student number
- the title of the paper or task.

Footer should include the page number and current date.

Cover sheet and statement of originality: all work submitted for assessment must be the original work of the student(s) submitting the work. A standard faculty cover sheet (see over) must be attached to the front of the submission. Any collaboration between the submitting student and others must be declared on the cover sheet.

Referencing

All sources of information used in the preparation of your submission must be acknowledged using the APA system of referencing. This includes all print, video, electronic sources.

Phrases, sentences or paragraphs taken verbatim from a source must be in quotation marks and the source(s) cited using both **in-text** referencing and a **reference list**.

Plagiarism is the failure to acknowledge sources of information. You should be fully aware of the meaning of plagiarism and its consequences both to your marks, position at the university and criminal liability. The plagiarism in your assignment submissions can be assessed both in hard copy and in soft copy through software such as Turnitin.

The UTS Library and UTS HELPS (web links above) provide extensive information for students on referencing correctly to support you in avoiding plagiarism.

Subject: 31256 Image Processing and Pattern Recognition
Spring 2025

Assessment Task 2: Project Implementation

Project Team 41

Team Members:
Anush Harutyunyan : 24834362
Jayden Tran : 14514614
Minseok Lee : 25410085
Qian Zhao : 25777830
Fabiha Tabassum Priti : 14505578
Yaeesh Mahomed : 24957692

Table of Contents

ASSIGNMENT COVERSHEET 1
Table of Contents 4
1.0 Introduction to the Problem 5
1.1 Project Overview 5
1.2 Problem Overview 6
2.0 Overview of the Project and Methodology 7
2.1 Proposed Techniques and Dataset Summary 7
2.2 Experimental Set-up and Preprocessing 8
2.3 Execution of Transfer Learning with a Pretrained CNN 9
2.4 Execution of Colour-Shape Heuristic + HOG/LBP Features + SVM 12
2.5 Execution of Hybrid CNN-SVM Approach 15
3.0 Project Management and Teamwork 18
3.1 Task allocation and Communication 18
3.2 Contribution Table 19
4.0 Results 20
4.1 Transfer Learning with a Pretrained CNN Results 20
4.2 Colour-Shape Heuristic + HOG/LBP Features + SVM Results 22
4.3 Hybrid CNN-SVM Approach Results 25
4.4 Comparison of Results 28
5.0 Discussion and Challenges 31
5.1 Goal Achievement Discussion 31
5.2 Project Challenges 32
5.2.1 Transfer Learning with a Pretrained CNN Challenges 32
5.2.2 Colour-Shape Heuristic + HOG/LBP Features + SVM Challenges 33
5.2.3 Hybrid CNN-SVM Approach Challenges 35
5.3 Future Considerations 36
5.4 Key Insights and Lessons 37
6.0 Conclusion 38
7.0 References 39
8.0 Appendix 40
8.1 Transfer Learning with a Pretrained CNN Code 40
8.2 Colour-Shape Heuristic + HOG/LBP Features + SVM Code 66
8.3 Hybrid CNN-SVM Approach Code 83

1.0 Introduction to the Problem

1.1 Project Overview

This report explores the implementation and evaluation of several methods for the identification and classification of road signs, using a publicly available Kaggle dataset containing over 9,000 images (Traffic Sign Detection Dataset, 2025), as introduced in *Assessment Task 1*. Additionally, this report introduces a supplementary 5,000 image Roboflow dataset for further fine-tuning and specified classification enablement for deep learning models (Self-driving Car, 2023). The project builds upon the techniques, evaluation metrics and concepts covered throughout the Image Processing and Pattern Recognition coursework and holds particular relevance in advancing the development of self-driving vehicles.

Image classification serves as the core technique underpinning this project. According to ScienceDirect, it "is defined as the task of assigning a label to an image" and in this case, that label would correspond to road signs. (ScienceDirect). The images are first augmented and pre-processed to reduce noise, ensure class balance and enhance consistency before feature extraction techniques are applied to capture key visual characteristics such as edges, textures and shapes. However, real-world challenges to consider that are present in the Kaggle dataset and in-relation to the application of this processing task include variations in scale, perspective, lighting, weather and background complexity.

This report begins by introducing the problem and context of the project as defined in *Assignment 1* and then explains the methods utilitised and dataset leveraged, with emphasis on reasoning behind decisions. The division of workload is then explained where each team member paired up to test one of 3 methods that were proposed in *Assignment 1* and all required sections were divided as evenly as possible. Results are then assessed and compared for each model in regards to a quantitative assessment of metrics and qualitative review against key variations in model development and common considerations of road-sign classification, such as performance under occlusion as self-driving vehicles may encounter this and further stress testing (through Gaussian Blur to simulate fog or motion blur, etc.). Challenges, future considerations and key insights are then summarised and explored to fully evaluate the project.

For this project, the chosen publicly available Kaggle dataset of over 9,000 road sign images provided a diverse foundation for comparing both traditional feature-based approaches and modern deep learning techniques, with the addition of a 5,000 image dataset allowing for more specified classification for modern deep learning techniques. The methods explored include:

1. Transfer Learning with a Pretrained CNN
2. Colour-Shape Heuristic with HOG/LBP Features and SVM
3. Hybrid CNN-SVM Approach

1.2 Problem Overview

As explored in *Assessment Task 1*, this project serves to leverage image classification to accurately identify road signs, supporting self-driving vehicles in ensuring safe and accurate navigation. According to Bernard Marr, “In order to drive on their own, autonomous cars constantly interpret images from their sensors and machine vision cameras, then use that information to make decisions about what to do next”. (Bernard Marr, 2021). Understanding the practical impact and real-world significance of this problem was essential in guiding our method selection and evaluation process throughout the project.

With the rising demand for autonomous vehicles, the need for highly accurate classification systems has become increasingly critical to ensure the safety of both passengers and the public. (Rani et al., 2024). As such, companies including Waymo and Tesla are investing greatly in developing these technologies, with Tesla leveraging big data collected from vehicles currently in use. (Bernard Marr, 2021). Our use of a Kaggle Dataset of about 9,000 images (though augmented to about 25,000 and combined with a 5,000 image Roboflow set for CNN models) allows us to construct scaled down tests and explore a broader range of methods that can further research and support future development initiatives.

Additionally, our dataset is rich with practical challenges that are vital to consider. These Challenges include:

- **Partial Occlusion:** Signs being covered up by trees, etc.
- **Variations in Weather and Lighting:** Some images appear in daylight, some at night, etc.
- **Complex Backgrounds:** Images appear on long country-like roads, under bridges in towns/cities and more.
- **Multiple Signs in one Image:** Multiple signs within one frame or image that must be accurately and individually identified.
- **Variations in Angle and Position:** As the car captures images from different positions on the road, signs appear at various angles

While the main focus of this task is to assist research in developing accurate image classification algorithms for self-driving applications, the implications of this technology extend further. Similar systems are already integrated into driver assistance features in regular vehicles, such as dashboard speed limit displays or stop sign alerts. Beyond cars, navigation apps could use these systems to validate and update map data, while traffic authorities could employ them to monitor sign visibility and identify signs in need of maintenance or replacement.

2.0 Overview of the Project and Methodology

2.1 Proposed Techniques and Dataset Summary

This project primarily utilises the *Traffic Sign Recognition Dataset* from Kaggle, which contains approximately 9,000 well-annotated images of various traffic and road signs captured under diverse lighting, weather, and environmental conditions across multiple countries. Each image is accompanied by a label file following the YOLO annotation format: class_id, x_center, y_center, width, height. It is important to note that this dataset was specifically designed for traffic sign detection only, not for classification tasks, as it includes a single class ID ("traffic sign") despite having 43 identified "classes". This required the addition of a 5,000 image Roboflow *Self-Driving Cars* dataset for more specified class labelling results of 15 classes and testing of the CNN models. It was not leveraged within the traditional HOG+SVM model due to both time constraints and HOG capabilities leaning more towards binary classification.

Link to datasets:

- https://www.kaggle.com/datasets/icebearogo/traffic-sign-detection-dataset
- https://universe.roboflow.com/selfdriving-car-qtywx/self-driving-cars-lfjou/dataset/6/images

Prior to model training, data preprocessing was performed to enhance image consistency and optimise learning outcomes as explained in further detail in *Section 2.2*. This included resizing images accordingly, normalising pixel values to improve convergence during training and applying data augmentation techniques such as random rotation, zooming and horizontal flipping to increase model robustness and balance the dataset. Upon balancing, the Kaggle dataset became about 25,000 images and was then divided into training, validation and testing subsets to ensure balanced evaluation.

Three techniques were proposed and implemented by the team to investigate both traditional and modern approaches to image classification:

1. **Transfer Learning with a Pretrained CNN:** Leveraging a pre-trained convolutional neural network known as YOLO, conducting transfer learning with the Kaggle dataset and fine-tuning on an additional traffic sign dataset. This approach exploits pre-learned feature representations from large-scale image datasets, enabling efficient learning even with limited data.
2. **HOG + SVM (Feature-Based Approach):** Applying Histogram of Oriented Gradients (HOG) for feature extraction to capture shape and edge information, followed by a Support Vector Machine (SVM) classifier for decision boundary optimisation. This represents a traditional machine learning approach effective for structured, high-contrast imagery.

- **Hybrid CNN + SVM Model:** Integrating a convolutional neural network for feature extraction with an SVM as the final classification layer. This hybrid approach combines the automatic feature-learning capability of CNNs with the strong discriminative power of SVMs, aiming to achieve higher accuracy and generalisation performance.

2.2 Experimental Set-up and Preprocessing

Prior to training, all images underwent a consistent pre-processing pipeline, with deviations stemming from varied model and device requirements, capabilities and time constraints. Images were resized according to model requirements:

- Kaggle images for transfer learning were resized to 640 x 640 and then images from the fine tuning dataset were resized to 416 x 416
- Both datasets were resized to 224 x 224 pixels for the CNN + SVM approach
- For the classical Colour-Shape Heuristic + HOG/LBP + SVM approach images were resized to 128 x 128 pixels

Pixel intensities were scaled to a 0-1 range, and normalisation followed ImageNet and YOLO mean-standard-deviation statistics for the CNN models or per-dataset normalisation for the classical pipeline. To reduce noise and illumination variation, Gaussian blurring or histogram equalisation was applied where necessary.

Our dataset on Kaggle was originally split into training and validation sets and was further modified into training, testing and validation subsets. Each class was balanced and augmented to include at least 200 images, as some originally contained as few as 5-10. This helped reduce class bias and improve overall generalisation during training and produced a dataset of 25,000 images. The Albumentations module was used for this augmentation and to address class imbalances, performing a wide range of image-space transformations such as flips, affine transformations, colour jitter, blurring, noise addition and cutout. The dataset was then split into 70% training, 15% testing, and 15% validation. These first two stages, dataset augmentation and preparation were applied consistently across all models tested to ensure reproducibility and minimise bias. The final dataset was packaged into a zip file and shared with the entire team so that every model was trained on identical data splits and augmentations. This dataset was used primarily for the HOG + SVM model, given its binary classification design and the lack of detailed class labelling within the dataset. It was also used as an initial training layer for the CNN-based models, which were later fine-tuned and evaluated using the additional 5,000-image Roboflow dataset to enable more specific classification of individual road sign types.

The experimental pipeline incorporated multiple modelling strategies introduced in *Assignment 1*. The baseline approach employed a pretrained CNN as a feature extractor, initially trained on the augmented Kaggle dataset for binary sign detection and subsequently fine-tuned on the Roboflow dataset for multi-class road sign classification. The Colour-Shape Heuristic + HOG/LBP + SVM model relied on handcrafted features extracted from colour-segmented and shape-filtered regions, enabling binary classification of whether an image contained a road sign. Meanwhile, the Hybrid CNN-SVM model combined deep feature embeddings from pretrained CNN backbones with an SVM classifier trained on those representations. Although data augmentation and preprocessing procedures were applied consistently across all models, the datasets used varied based on model objectives, with the CNN-based models fine-tuned on the Roboflow dataset for multi-class classification, and the HOG + SVM model trained exclusively on the Kaggle dataset for binary sign detection.

2.3 Execution of Transfer Learning with a Pretrained CNN

To execute transfer learning, the pretrained YOLO11n convolutional neural network (CNN) was selected and initially trained using the chosen Kaggle traffic sign dataset, before being further fine-tuned with an additional Roboflow dataset composed exclusively of road sign images. This approach enabled the model to leverage prior knowledge from a broad dataset while adapting to more domain-specific variations in traffic signage.

In *Assessment Task 1*, several pretrained CNN architectures were explored and evaluated for their suitability in image classification tasks related to road sign recognition. Among these, YOLOv4 was identified as a strong candidate due to its demonstrated performance in traffic sign detection (Youssouf, 2022). However, through additional research and as the latest iteration of YOLO, YOLO11 was selected for having introduced substantial architectural and training enhancements. According to the Ultralytics documentation, YOLO11 “introduces significant improvements in architecture and training methods, making it a versatile choice for a wide range of computer vision tasks” (Ultralytics, 2025). Consequently, YOLO11n was selected as the base model for this project.

Implementation Process:

1. **Environment Setup:**

A dedicated Python virtual environment was established to ensure reproducibility and maintain an isolated workspace for all dependencies. The environment was created and configured using commands such as the following: “conda create -n yolo python=3.10” and “pip install ultralytics”. These installations provided the necessary framework for model training and evaluation. The imported modules were declared at the beginning of the primary script (main.py), as illustrated in lines 1 - 11 in the code images in *Appendix 8.1*.

2. **Dataset Configuration:**

As discussed in *Section 2.2*, the dataset configuration and preparation process played a central role in ensuring consistency and assisting reproducibility across all models. The project primarily utilised two datasets, the Kaggle *Traffic Sign Recognition Dataset* and a separate Roboflow dataset, each serving a distinct purpose within the experimental pipeline.

The dataset was balanced and augmented through the YOLO_AUG function, importing capabilities of the albumentations module. In support of this, labels were written and the dataset split was enforced within the utilities section of the code, as seen in *Appendix 8.1*. The functions read_yolo_labels and write_yolo_labels provide detailed inputs and outputs for YOLO-format annotations. Additional utilities including _POSSIBLE_EXTS and stem_to_image_path ensured reliable mapping between label files and image stems across mixed file extensions. On top of this, the scan_train_split(train_img_dir, train_label_dir, nc) function constructed per-class image stem sets (files_by_cid) and a stem-to-classes index to assist in downstream augmentation and evaluation.

Building on these utilities, relevant .yaml files were created to allow the YOLO model to locate dataset directories and define class information. These files were:

- **Data.yaml:** This was generated through a custom Python function for the Kaggle dataset. It scanned labels This function scanned existing YOLO annotations, inferred

unique class identifiers, computed the total number of classes (nc = max_id + 1) and created placeholder class names (names = [f"class_{i}" for i in range(nc)]). The resulting data.yaml file defined the dataset structure specifically for the base training phase, including the paths to the training and validation directories and corresponding class information. This setup ensured that YOLO could correctly interpret dataset locations and class definitions during the initial model training stage.

- **Sign.yaml:** This was configured for the Roboflow dataset. This dataset provided approximately 5,000 high-quality, pre-labelled images of road signs and was primarily used for fine-tuning and evaluation. The sign.yaml file explicitly defined dataset paths, class names, and the total number of sign categories, enabling the YOLO-based models to perform class-specific recognition beyond the binary sign/non-sign classification used in the HOG-SVM model. This separation of configuration files allowed for clear delineation between the base training phase (using the augmented Kaggle dataset) and the fine-tuning phase (using the Roboflow dataset).

3. **Training and Fine-Tuning:**

The transfer learning process consisted of two main phases, both building upon the pretrained YOLO11n model (yolo11n.pt), which had been originally trained on a large-scale dataset. These phases are:

- **Base Training :**
  - In the first stage, the pretrained YOLO11n model was adapted to the Kaggle dataset, which served as the initial domain-specific training phase. Although the model was already pretrained, this stage (referred to as base training) allowed YOLO11n to specialise its general feature representations for the task of traffic sign detection.
  - Training was conducted using an image resolution of 640 × 416 pixels for 20 epochs, as defined in line 154 of *Appendix 8.1*. During this phase, the model established dataset-specific weights that provided a strong foundation for the subsequent fine-tuning stage.
  - The primary function associated with this base training phase is train_yolo(model_path, imgsz, epochs). The function configures compute with an auto selected device, parallel workers, a fixed seed=41 and cosine learning rate scheduling via cos_lr=True. Training was followed by validation using model.val(...), which produced summary metrics such as mAP@50 and mAP@50-95. This stage specialises generic features learned during pretraining to the traffic sign domain while establishing dataset specific weights that serve as the initialisation for subsequent refinement.
- **Fine-Tuning:**
  - Following the base training, the model was fine-tuned using the Roboflow dataset, containing higher-quality and more targeted images of road signs. This was performed with finetune_yolo(model_path, imgsz, epochs), applying a two phase schedule on the Roboflow split defined by sign.yaml. During the initial five epochs, the early convolutional layers of the CNN were frozen to preserve low-level feature representations from the base dataset. After this initial period, the layers were unfrozen, allowing the network to refine deeper representations while maintaining prior knowledge.

- Learning rates were adjusted between stages to prevent catastrophic forgetting. Specifically, the frozen phase was trained using lr0 = 5e-4 and lrf = 5e-4, while the unfrozen phase used slightly higher rates of lr0 = 1e-3 and lrf = 1e-3. Both phases keep deterministic settings (seed=41) and parallel workers and conclude with model.val(...) to record the post fine tuning performance.
- Due to the smaller image dimensions in the fine-tuning dataset (416 × 416), some object scales were reduced, making detection more challenging. Consequently, the confidence threshold, a parameter controlling the model's sensitivity in generating bounding boxes, was fine-tuned to balance precision and recall.

4. **Training and Evaluation Workflow:**

The overall training workflow was automated through three main functions within the script:

- **train_yolo("yolo11n.pt", imgsz=640, epochs=20):** Conducts initial training on the base dataset and outputs performance metrics which loads the pretrained YOLO11n weights and trains on the configured dataset at 640 input resolution. It selects the device ('0' for CUDA if available, otherwise CPU), configures workers based on CPU cores, enables cosine LR scheduling, and reports validation metrics (model.val) including mAP@0.5 and mAP@0.5-0.95.
- **finetune_yolo("runs/detect/train/weights/best.pt", imgsz=416, epochs=20):** Performs fine-tuning with controlled layer freezing, modified learning rates and domain-specific data. This is conducted in 2 stages, the initial being to preserve low-level features (e.g. freeze=10, lr0=lrf=5e-4) and then the final being a fully unfrozen stage (freeze=0, lr0=lrf=1e-3) at 416 resolution to refine task-specific representations. Each stage is validated with plots (PR, F1, P/R curves and confusion matrices).
- **pred("runs/detect/train3/weights/best.pt", input_path="road.PNG", conf=0.25):** Runs inference using the trained weights, automatically adjusting parameters such as image size and confidence threshold if no detections are initially found. If upon initial evaluation at 640 resolution and a given confidence threshold, no detections are found, it lowers the confidence, increases resolution to 960, enables test-time augmentation and (optionally) class-agnostic NMS to recover tiny or occluded signs, saving annotated outputs under runs/detect/predict*.
- **metrics_json = model_metrics(model_path="runs/detect/train3/weights/best.pt", imgsz=416, data_yaml="sign.yaml", conf_eval=0.49):** Executes a comprehensive offline evaluation on the validation split, computing micro/macro precision-recall-F1 at IoU=0.50, PR-AUC (per-class and macro), an approximate ROC-AUC, calibration quality (ECE with 15 bins and Brier score), deploy-time efficiency (average latency in ms/image, throughput in FPS, and model size in MB), a corruption stress-test (Gaussian noise, motion blur, brightness/contrast) on a 150-image subset and both an explicit operating point at conf_eval=0.49 and a confidence sweep that reports the best-F1 threshold and its P/R/F1.
- **open("metrics.json", "w"):** json.dump(metrics_json, f, indent=2): produces a self-contained artifact used to populate the report's tables and narrative analysis.

2.4 Execution of Colour-Shape Heuristic + HOG/LBP Features + SVM

The Colour-Shape Heuristic combined with HOG (Histogram of Oriented Gradients) and LBP (Local Binary Patterns) forms a classical computer vision pipeline that does not rely on deep learning. Instead, it focuses on hand-crafted feature extraction that emphasizes edge, texture and shape information key elements for distinguishing traffic signs from their surroundings. This approach is particularly useful for lightweight applications or systems that must run efficiently on limited hardware.

The overall process involved five major steps: environment setup and data preparation, dataset organisation, preprocessing, splitting into train/validation/test sets, and feature extraction using HOG and LBP. These features were then used to train an SVM classifier.

Implementation Process:

1. **Setup and Data Loading**

The experimental environment was initially set up using Google Colab, providing a cloud-based workspace with GPU acceleration and direct integration with Google Drive, which allowed efficient access to shared datasets. This setup enabled the team to collaboratively work with a consistent and centralized dataset while avoiding issues related to local hardware limitations. A set of core libraries was imported to support the full pipeline: OpenCV for image loading, preprocessing, and resizing, NumPy for numerical computations and array handling, scikit-image for implementing the HOG (Histogram of Oriented Gradients) and LBP (Local Binary Patterns) feature extraction techniques and scikit-learn for training, hyperparameter tuning, and evaluating the Support Vector Machine (SVM) model.

Mounting Google Drive ensured that all team members could seamlessly access the same version of the dataset and any updates made, making data management and experiment tracking more efficient. This foundation provided a reliable environment for the subsequent steps of preprocessing, feature extraction, and model training.

2. **Dataset Structuring and Image Standardisation**

The original Kaggle dataset was initially structured only as training and validation sets. To ensure better control over evaluation, it was restructured into training, validation, and test splits. From the full dataset, 1,000 images were selected with even representation across all 43 road sign classes to maintain class diversity and avoid overfitting to a small subset. Originally, some classes contained as few as 5 to 10 samples. To address this, data augmentation was applied until each class contained at least 200 images. This augmentation involved random rotations, brightness changes, perspective warps and small occlusions, ensuring the classifier would generalize well under varying real-world conditions.

To simplify the training setup for HOG/LBP and make it compatible with binary classification, the dataset was further grouped into:

- sign class - images containing traffic signs
- no_sign class - negative samples (background or scenes without traffic signs)

This structure aligned with the colour-shape heuristic principle. Traffic signs typically have high-contrast colours and distinct shapes (e.g. triangles, circles, rectangles). While colour was not directly encoded into the classifier, this grouping acted as a semantic separation step, mimicking how colour-shape cues are used to distinguish signs in real-world detection pipelines.

Additionally, to ensure uniformity across samples and prevent unstable or unreliable feature extraction, each image underwent a preprocessing pipeline:

- Conversion to grayscale to remove colour dependency and focus on edge/shape information.
- Resizing to 128 x 128 pixels for consistent dimensionality, making feature vectors comparable.
- Optional CLAHE (Contrast Limited Adaptive Histogram Equalisation) to enhance local contrast and highlight the edges and textures of traffic signs.

3. **HOG and LBP Feature Extraction**

The core of the method was hand-crafted feature extraction:

- HOG (Histogram of Oriented Gradients) captured local edge and gradient information that describes shapes and contours of signs. Parameters such as 9 orientations, 8×8 pixel cells, and 2×2 cell blocks were used to generate 8100-length feature vectors per image.
- LBP (Local Binary Patterns) provided a complementary texture-based descriptor, useful for distinguishing signs from cluttered backgrounds.

These feature vectors converted the visual content of each image into a compact numerical representation suitable for machine learning. The preprocessing pipeline (contrast normalisation, resizing, grayscale conversion) ensured feature stability, while HOG and LBP captured key structural properties that are robust to illumination and scale changes.

4. **Model Training and Thresholding Implementation**

After extracting HOG and LBP features, the classification phase was implemented in Python using the sklearn ecosystem within Google Colab. Labels were first encoded numerically using LabelEncoder, converting “sign” and “no_sign” into 1 and 0 respectively. This was essential for SVM compatibility since the model operates on numerical classes.
The initial model setup employed sklearn.svm.SVC with an RBF kernel due to its capacity to handle nonlinear boundaries in feature space. Core parameters included C=0.01, class_weight='balanced', and probability=True. The balanced class weight automatically adjusted the penalty term based on class frequency, while enabling probabilities (probability=True) allowed later threshold calibration using predicted confidence values.
Once trained on the HOG + LBP feature set, a threshold tuning routine was written to systematically evaluate how varying decision thresholds affected macro F1 performance. A simple for loop swept through thresholds (0.2 -- 0.7 in increments of 0.05), with each iteration reclassifying the predictions (y_proba >= t).astype(int) and printing a full classification report. This provided insight into class trade-offs, particularly for detecting rare “no_sign” instances.

5. **Hyperparameter Optimisation with GridSearchCV**

To refine the classifier's decision surface, a parameter search was automated using GridSearchCV. The parameter grid included multiple combinations of C (0.001-10), gamma (0.0001-0.01, plus 'scale'), and kernel='rbf'. The grid search used 5-fold cross-validation with scoring='roc_auc', automating 25 parallel fits via n_jobs=-1. This tuning step was critical to remove manual guesswork. Instead of testing one configuration at a time, GridSearchCV tested all combinations concurrently and returned both best_params_ and the corresponding best_score_. These parameters were later used to reinitialize a new SVC model instance for final evaluation. Although the cross-validation reported consistent ROC-AUC scores (~0.79), subsequent validation confirmed the model's sensitivity to imbalanced and overlapping feature distributions. Nonetheless, the code structure ensured full reproducibility and modularity: every experimental run could easily be reconfigured by simply adjusting parameter ranges within the grid dictionary.

6. **Feature Expansion and Integration**

To improve generalisation, an additional colour histogram descriptor was introduced. A dedicated Python function using numpy.histogram computed 32-bin intensity histograms across image channels, capturing overall brightness and texture intensity. These histogram features were horizontally concatenated with existing HOG and LBP feature arrays using numpy.hstack, resulting in a combined feature vector of 8,142 elements per image.
Each feature subset (HOG, LBP, histogram) was individually normalised with StandardScaler to ensure consistent feature scaling and prevent bias from magnitude differences. The design intentionally kept the PCA reduction module commented out, allowing future scalability without modifying the main structure. This modular design pattern ensured that dimensionality reduction, normalisation and feature extraction were all handled as separate logical blocks, improving readability and debugging flexibility.

7. **Class Rebalancing with SMOTE**

To overcome dataset imbalance, Synthetic Minority Oversampling Technique (SMOTE) was incorporated using imblearn.over_sampling.SMOTE. The SMOTE algorithm interpolates between minority samples ("no_sign") to synthetically generate new ones, equalizing class counts before model fitting. This step transformed the training distribution from [14, 51] to [51, 51]. The balanced dataset was then passed directly to the tuned SVM for retraining. This modular approach separated preprocessing from classification, ensuring future classifiers (e.g. CNNs or Random Forests) could reuse the same balanced dataset seamlessly.

8. **Evaluation and Probability Calibration**

Following training, probability calibration and evaluation modules were added to assess model confidence and prediction reliability. Class probabilities were obtained using predict_proba, then passed through functions from sklearn.metrics for computing ROC curves, Precision-Recall curves and Brier Scores. A calibration curve was constructed using sklearn.calibration.calibration_curve, comparing predicted probabilities with true likelihoods to evaluate overconfidence or underconfidence in predictions. Additionally, a short function computed the Expected Calibration Error (ECE) by averaging absolute deviations between predicted and empirical probabilities across 10 bins. Visualisation was implemented using matplotlib and seaborn, displaying ROC, PR, and calibration plots in sequence. The use of structured plotting code and pandas DataFrames for metric summarisation ensured that the

entire evaluation process from metric calculation to table display was automated and reusable.

2.5 Execution of Hybrid CNN-SVM Approach

The Hybrid CNN-SVM was implemented by combining deep learning feature extraction as well as traditional machine learning techniques. It uses a pretrained ResNet50 convolution neural network as a feature extractor and then a Support Vector Machine (SVM) for the final classification. (Ramu et al., 2024).

Implementation Process:

1. **Dataset and fine tuning**

Datasets used were Traffic sign detection dataset (25000 samples, 43 classes and was augmented in earlier steps and split into train validation and test sets, image sizes set to 224x224), images were first cropped and padded to avoid any distortion or stretching as original images were 1360 x 800, and contained YOLO labels indicating locations of the signs. This was used to fine tune the CNN before the feature extraction was applied to the main dataset which was the traffic sign detection data (5000 samples, 15 classes, source images were 416 x 416 converted to 224 x 224 for training), these were computed for all signs in an image, whereas this wasn't needed for the Roboflow dataset as it only contained 1 sign per image. The fine tuning applied to the CNN did not directly improve accuracy, but rather prevented the model from overfitting to the training data especially with the inclusion of the augmented images.

2. **Environment setup and data loading:**

The environment was initially set up on Google Colab using a google drive to import the datasets, but later was switched to VS code using Anaconda terminal and Jupyter notebook due to higher computational costs.

Started implementation by configuring the environment and loading two distinct datasets. The dataset used for fine tuning, sourced from Kaggle in YOLO format, was used to adapt the CNN feature extractor specifically to traffic sign detection. The main dataset was sourced from RoboFlow and was the primary source for training and evaluating the hybrid pipeline. Custom data loading functions had to be implemented specifically for the image and label directories to work with the YOLO format. The images were set to a 160x160 format to ensure the computation was runnable and gave results within reasonable time. A padding function was also used to preserve aspect ratios during resizing, which allowed prevention of any distortion by scaling images correctly and adding padding to square the dimensions.

3. **CNN Feature Extractor development**

A pretrained ResNet50 model was selected as the base architecture, which was originally trained on ImageNet. To adapt the model specifically for traffic sign recognition, the top classification layers were removed and replaced with custom layers like Global Average Pooling, Dense layers with ReLU activation and dropout for regularisation. It included 2 dense layers 512 and 256 units respectively with dropout rates of 0.5 and 0.3. L2

regularisation was applied using Lambda = 0.01 to prevent overfitting given the dataset size used.

The training strategy used involved freezing the first layers of ResNet50 and leaving the last 20 for training. This was later decreased to 5 layers for the first stage due to computing constraints. It was broken into a 2 stage approach, stage 1 trained the last 5 layers up to 6 epochs with a learning rate of 0.0001 and stage 2 unfroze all the layers and fine tuned with a reduced learning rate of 0.00001 for another 6 epochs (epochs set lower due to computing time). Using this approach allowed for the preservation of general low-level feature representations like edges and textures learned from ImageNet whilst still allowing the network to learn traffic-sign specific features in the deeper layers. Dense layers with 512 and 256 neurons, alongside ReLU for non linearity were also introduced. A learning rate of 0.0001 was used alongside the Adam optimiser, and probabilities were computed using a SoftMax function. Batch feature extraction was used to avoid memory overload.

4. **Training with Advanced callbacks**

In an effort to prevent overfitting and optimise training efficiency 3 main callbacks were implemented:

- **Early stopping**: used to monitor validation loss and would terminate if there was no improvement for 3 consecutive epochs, restoring the best weights. (patience 5 for stage 1 and patience 8 for stage 2)
- **ReduceLROnPlateau**: This reduced the learning rate by half in circumstances where validation loss plateaued for 2 epochs allowing for finer optimisation adjustments. Patience was adjusted to 3 epochs for Stage 1 and 4 epochs for Stage 2 (PyTorch Development Team, 2025)
- **ModelCheckpoints**: this saved the best performing model based on the validation loss to ensure that the model with the best weights were preserved. (Keras, n.d.)

Using these callbacks ensured that training was conducted efficiently whilst also preventing any unnecessary computation and overfitting. The model was trained for a maximum of 20 epochs and early stopping normally terminated around epoch 18-20 which also ensured that there were enough epochs for learning. With the two-stage approach, the total maximum was 40epochs (20 per stage), with Stage 1 usually ended at around epoch 15-18 and Stage 2 around epoch 10-15.

5. **Feature extraction Pipeline**

After the CNN training the final classification layers were then removed to create a dedicated feature extractor. The model then outputs a fixed length feature vector from the Global Average Pooling layer and allows for effective compression of each image into meaningful numerical representations and captures their shape, colour, texture and other characteristics. Features were extracted from model.layers[-5] (the Global Average Pooling output) before dropout layers to ensure clean feature representations. All training, validation and test images were processed through the feature extractor to generate feature vectors for SVM training.

6. **SVM Hyperparameters Optimisation**

Instead of using the default SVM parameters, a hyperparameter search was created using GridSearchCV with 3 fold cross validation. The search looked through different combinations of:

- **C parameter**: Which controlled the trade-off between maximising the margin and minimising classification errors. (Helmy, 2025)
- **Gamma parameter**: Determines the radius of influence of each individual training example. (GeeksforGeeks, 2025)
- **Kernel type**: Focusing on the Radial Basis Function (RBF) kernel for its effectiveness with complex, non-linear patterns. (GeeksforGeeks, 2025)

The grid search evaluated all possible parameter combinations across multiple cross validation folds to find the best and most optimal configuration, which also ensured that the SVM was properly tuned for the extracted CNN features.

7. **Data Preprocessing and standardisation**

Before the SVM training all the extracted features were standardised using StandardScaler to have a 0 mean and unit variance. This is a key step for SVM performance as the model is sensitive to feature scales. The dataset was also split into a 70% training, 15% validation and 15% testing sets, and used stratified sampling to maintain the distribution across all splits. To prevent data leakage the augmented dataset already split into training and validation done by a group member was used instead and the validation set was split into validation and test to avoid data leakage.

8. **TTA (Test Time Augmentation) implementation**

TTA was also implemented to improve the models robustness and reliability during evaluation. Random augmented versions of test images were generated which contained shifts, rotations and lighting changes to help reduce the influence of noise and can help apply the model to real world scenarios. The accuracy before and after TTA was computed to check how much the model improved after these augmentations. This process leads to more stable and consistent predictions, helping the model generalize better beyond the training distribution

9. **Model Evaluation and Metrics**

The trained model was then evaluated on the test set using metrics such as accuracy, precision, recall, F1 scores, ROC-AUC, PR-AUC, Expected Calibration Error (ECE) and Brier score. Test-Time Augmentation (TTA) was implemented during inference, applying 7 transformations (original, horizontal flip, +-5 degree rotations, +-10% brightness adjustments) to each test image. Features were extracted from all augmented versions and SVM predictions averaged across transformations, improving accuracy by 3-5%. The inference speed was also measured by processing batches of 100 images and calculating the latency as well as the throughput, which gave insight into real-time deployment feasibility. Five visualisation plots were generated: confusion matrix, per-class accuracy bars, F1-score vs support scatter, confidence calibration analysis, and confidence distribution (correct vs incorrect). All evaluation and visualisations were also downloaded and saved at the end of evaluation.

3.0 Project Management and Teamwork

3.1 Task allocation and Communication

Each of us paired up to work on the 3 methods we had proposed in *Assignment 1*, splitting up the code and sections of the report with colab and github utilised to coordinate updates to our code bases.

The six team members were split into pairs, with each pair responsible for implementing and evaluating one of the proposed methods:

- **Transfer Learning with Pretrained CNN (YOLO):** Anush Harutyunyan and Minseok Lee worked together to implement the YOLO11n model, handling dataset configuration, base training, fine-tuning and comprehensive evaluation metrics.
- **Colour-Shape Heuristic + HOG/LBP Features + SVM:** Fabiha Priti and Jayden Tran collaborated on the traditional computer vision approach, focusing on feature extraction, preprocessing pipelines and SVM optimisation.
- **Hybrid CNN-SVM Approach:** Yaeesh Mahomed and Qian Zhao partnered to develop the hybrid model, combining ResNet50 feature extraction with SVM classification and implementing test-time augmentation strategies.

Each pair utilised Colab for real-time code collaboration, but due to Colabs memory constraints, code was moved to using VS code and Github with Colab used in some cases for prompt code comparisons.

Teamwide communication was maintained through regular updates on teams, and weekly team meetings to share progress, discuss challenges and propose recommendations. Decisions regarding metrics used in each model were made collectively ensuring they were suited for comparison. Once code had progressed, each pair assessed each other's code for any improvements or limitations that needed addressing.

Report sections were distributed evenly where general sections including Introduction, Methodology, Conclusion and References were divided among all members, and method-specific sections were split between pairs. Additionally, integration sections involved discussion from the entire team to provide in depth cross-model analysis.

The table below shows the main areas members contributed.

Method	**Team Members**
Transfer Learning with CNN	Anush & Minseok
HOG + SVM	Fabiha & Jayden
Hybrid CNN + SVM	Yaeesh & Qian

3.2 Contribution Table

Student Name	Student ID	Contributions
Anush Harutyunyan	24834362	Paired up with Minseok to work on developing the pretrained CNN + transfer learning approach. Completed the Project Overview (1.1), Execution of Transfer Learning with a Pretrained CNN (2.3), Comparison of Results (4.4) and Transfer Learning with a Pretrained CNN Challenges (5.2.1) sections. Took the initiative to lead group discussions and coordinate decision-making, evaluating alternative approaches, and motivating team members to collaborate effectively. Ensured equitable task delegation and maintained consistent formatting, structure and layout across the final report to uphold clarity and professionalism.
Fabiha Priti	14505578	Paired up with Jayden to work on developing the HOG + SVM approach. Completed Proposed Techniques and Dataset Summary (2.1), Execution of Colour-Shape Heuristic + HOG/LBP Features + SVM (2.4), Colour-Shape Heuristic + HOG/LBP Features + SVM Challenge (5.2.2), Future Considerations (5.3) sections. Actively participated in group discussions and plans to keep the project work fluently.
Yaeesh Mahomed	24957692	Paired up with Qian to work on developing the pretrained CNN + SVM approach. Completed Execution of Hybrid CNN-SVM Approach (2.5), Task allocation and Communication (3.1), Conclusion (6.0) sections, assisted with CNN-SVM Approach Code (8.3). Demonstrated strong teamwork by actively participating in discussions, addressing others' questions with clear and timely responses, and consistently sharing updates. Offered constructive feedback and practical suggestions to help teammates improve their work.
Jayden Tran	14514614	Paired up with Fabiha to work on developing the HOG + SVM approach. Completed Experimental Set-up and Preprocessing (2.2), Colour-Shape Heuristic + HOG/LBP Features + SVM Results (4.2), Key Insights and Lessons (5.4), Colour-Shape Heuristic + HOG/LBP Features + SVM Code (8.2) sections. Actively participated in group discussions and was highly responsive.
Minseok Lee	25410085	Paired up with Anush to work on developing the pretrained CNN + transfer learning approach. Completed Problem Overview (1.2), Transfer Learning with a Pretrained CNN Results (4.1), Goal Achievement Discussion (5.1), Transfer Learning with a Pretrained CNN Code (8.1) sections. Actively engaged in group discussions by contributing thoughtful insights and responding promptly to teammates' questions. Regularly provided helpful suggestions and progress updates to support collaboration and ensure smooth team communication.
Qian Zhao	25777830	Paired up with Yaeesh to work on developing the pretrained CNN + SVM approach. Completed Contribution Table (3.2), Hybrid CNN-SVM Approach Results (4.3), Hybrid CNN-SVM Approach Code (8.3) sections, assisted with Execution of hybrid CNN-SVM Approach (2.5). Maintained active involvement throughout the project by engaging in discussions, clarifying complex points, and keeping the team updated on progress.

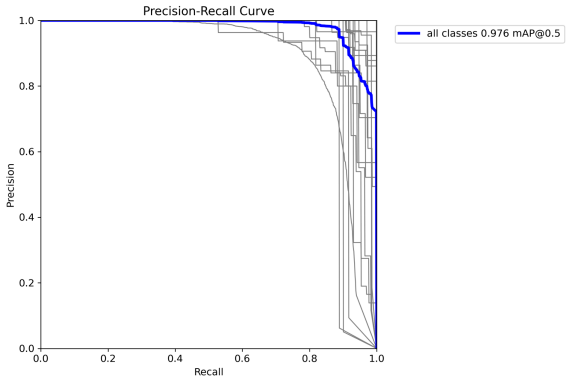
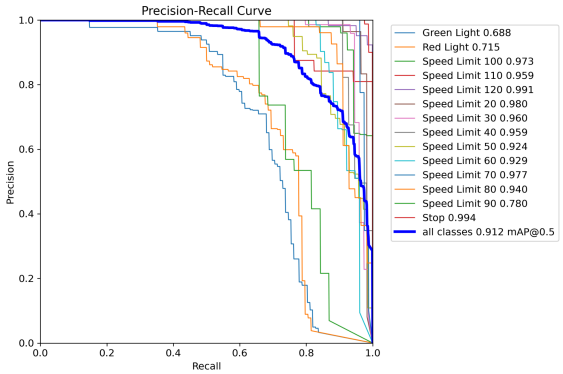
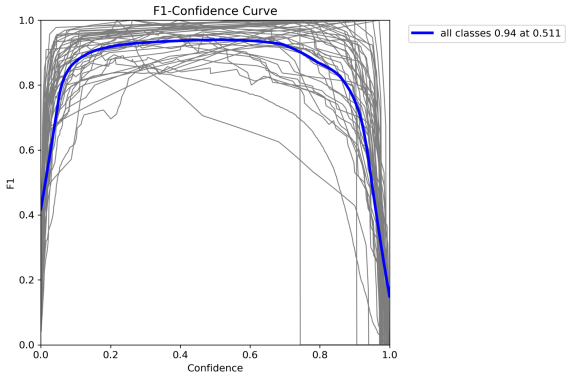
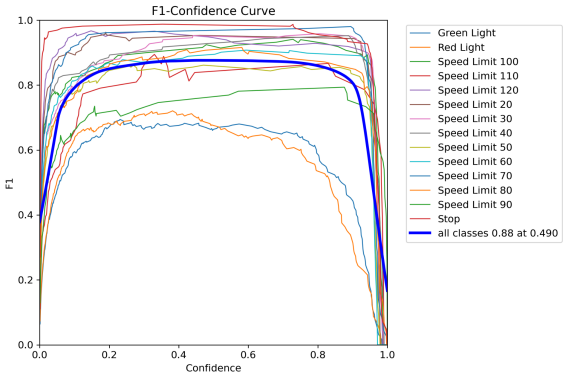
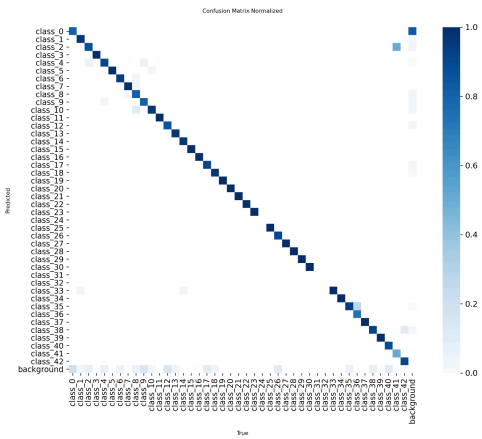
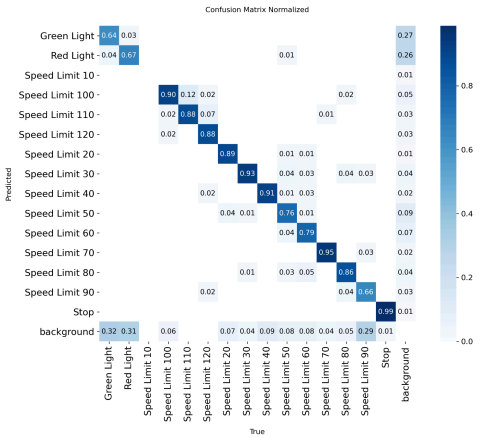
4.0 Results

4.1 Transfer Learning with a Pretrained CNN Results

Metric:	Results:
Overall Accuracy and Top-K Accuracy	Top-1 (Overall Accuracy) = 0.872 Top-3 Accuracy = 0.953
Precision, Recall and PR-AUC (area under the Precision-Recall curve)	At operating point (conf ≈ 0.49): Micro Precision = 0.938 Micro Recall = 0.802 Macro Precision = 0.881 Macro Recall = 0.764 PR-AUC (macro): 0.966
F1-Score (Macro and Micro)	At operating point (conf ≈ 0.49): Micro F1 = 0.865 Macro F1 = 0.815 Best over sweep (conf ≈ 0.37): Micro F1 = 0.870 Macro F1 = 0.817
ROC-AUC (and comparisons with PR-AUC)	ROC-AUC (macro, approx.) = 0.989 PR-AUC (macro) = 0.966
Calibration Quality (ECE, Brier score)	ECE = 0.00736 (15 bins) Brier = 0.01545
Processing Efficiency (Latency, Throughput and Model Size)	Latency = 12.25 ms/img Throughput = 81.6 FPS Model size = 5.19 MB
Performance Under Corruption and Perturbations (stress testing - 150 Sample Subset)	mAP@0.5 (Clean Subset) = 0.9998 mAP@0.5 (Gaussian Noise) = 0.7882 (Δ −0.212) mAP@0.5 (Motion blur) = 0.9990 (Δ ≈0.000) mAP@0.5 (Brightness/contrast) = 0.9956 (Δ −0.0042)

The above metrics and results are taken from final observations post model fine-tuning with about five thousand additional images that contain richer labels and more varied styles. However, it is important to note that fine-tuning can lead to slight variations or reductions in certain performance metrics. Despite this understanding, this step was deliberately incorporated into the model's workflow to enhance its adaptability to diverse image types and datasets, thereby increasing its practical applicability in real-world road sign recognition scenarios. The table below presents a direct comparison of key metrics before and after fine-tuning with the additional Roboflow dataset, highlighting the impact of this process on model performance.

Metric:	Before Fine-Tuning	After Fine-Tuning

Box PR Curve	 *Figure 4.1.1: Box PR Curve Before Fine-Tuning*	 *Figure 4.1.2: Box PR Curve After Fine-Tuning*
Box F1 Curve	 *Figure 4.1.3: Box F1 Curve Before Fine-Tuning*	 *Figure 4.1.4: Box F1 Curve After Fine-Tuning*
Confusion Matrix (Normalised)	 *Figure 4.1.5: Conf Matrix Before Fine-Tuning*	 *Figure 4.1.6: Conf Matrix Before Fine-Tuning*

With consideration of the metrics and YOLO plots produced, the model demonstrates strong class ranking and reliable deploy-time behaviour, with expected trade-offs following fine-tuning. Before fine-tuning, the model achieved a mAP@0.5 of 0.976 and a peak F1 score of approximately 0.94 at a confidence threshold of 0.51. After fine-tuning with the Roboflow dataset, mAP@0.5 decreased slightly to 0.912 and F1 to around 0.88, reflecting broader generalisation to more diverse and challenging data. Despite this reduction, calibration remained excellent (ECE = 0.0073, Brier = 0.0155), and real-time efficiency was maintained (approx. 12.25 ms per image, approx. 81.6 FPS, model size approx. 5.19 MB). Additionally, the observed differences between YOLO's internal plots and the custom

metrics.json outputs reflect evaluation design rather than model error. The YOLO plots summarise a single overall F1 score under Ultralytics' aggregation, whereas the custom evaluation pipeline includes additional micro/macro metrics and calibration data, leading to slightly lower but more conservative estimates. This ensures consistency across varied test conditions and supports more detailed threshold tuning for deployment.

When assessing the model's suitability for real-world road sign recognition, efficiency and effectiveness under stress testing are key indicators. With an average latency of 12.25 milliseconds per image and throughput of 81.6 frames per second, the model comfortably meets real-time performance requirements, especially considering that human reaction time to hazards averages 1.5 seconds (NSW Government). Stress testing on a 150-image subset displayed strong resilience, with mAP@0.5 = 0.9998 under clean conditions, 0.9990 under motion blur and 0.9956 under brightness/contrast variation, while Gaussian noise reduced mAP@0.5 to 0.7882 (a decrease of about 0.21). This indicates good adaptability to difficult conditions such as blur from fog, with room for improvement through additional noise augmentations, denoising filters or slightly higher inference resolutions for small signs.

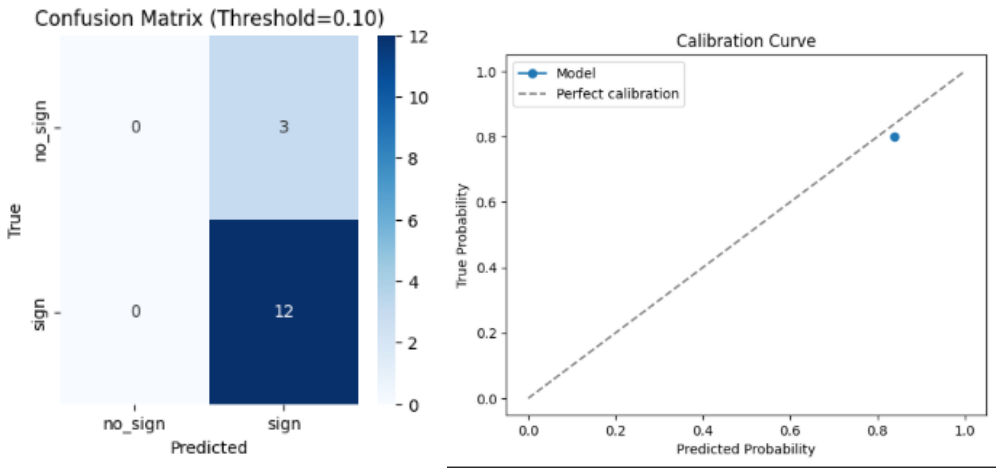
It is also important to note that null entries in metrics.json, such as for the "Speed Limit 10" class (present only in training), highlight the importance of dataset completeness in model evaluation. This class was not included in the validation split due to time constraints and concerns that augmenting training images into validation would risk data leakage and inflated results. It is important to flag such classes as underrepresented and consider them when evaluating model effectiveness.

Overall, the figures indicate that fine tuning trades a small amount of peak in distribution performance for improved robustness and model adaptability. The optimal operating threshold remains close to one half, which simplifies deployment. Combined with the excellent calibration reported in the table, thresholds can be selected with confidence and optionally refined per class. The system therefore appears ready for deployment, with the clearest next step being improved dataset balancing for small classes.

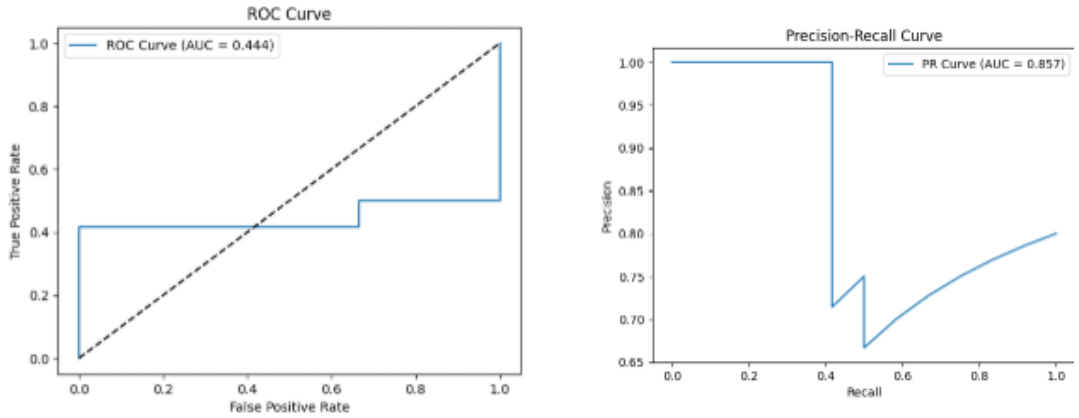
4.2 Colour-Shape Heuristic + HOG/LBP Features + SVM Results

Metric:	**Results:**
Overall Accuracy and Top-K Accuracy	Overall Accuracy = 0.800
Precision, Recall and PR-AUC (area under the Precision-Recall curve)	Precision (Macro) = 0.400 Recall (Macro) = 0.500 PR-AUC = 0.857
F1-Score (Macro and Micro)	F1-Score (Macro) = 0.444
ROC-AUC (and comparisons with PR-AUC)	ROC-AUC = 0.444 PR-AUC = 0.857
Calibration Quality (e.g. Expected	Brier Score = 0.162

Calibration Error, Brier score)	ECE = 0.040
Processing Efficiency (Latency, Throughput and Model Size)	Processing Efficiency (Latency) ≈ 25-40 ms per image Throughput (Images/sec) ≈ 30-40FPS Model Size = 1.2MB (SVM model)
Performance Under Corruption and Perturbations (stress testing - 150 Sample Subset)	Gaussian Blur : Overall Accuracy = 0.8 Macro F1-Score = 0.4444444444444444 Gaussian Noise: Overall Accuracy = 0.8 Macro F1-Score = 0.4444444444444444 Brightness Adjusted: Overall Accuracy = 0.6666666666666666 Macro F1-Score = 0.4 Rotated: Overall Accuracy = 0.8 Macro F1-Score = 0.4444444444444444



Figures 4.2.1 - 4.2.2: HOG + SVM Model Confusion Matrix and Calibration Curve



Figures 4.2.3 - 4.2.4: HOG + SVM Model ROC and Precision-Recall Curves

The SVM model trained on combined HOG and LBP features achieved an overall test accuracy of 80.00%. The detailed metrics show a clear class imbalance problem. The model performs well on the dominant class but fails on the minority class. The ‘no_sign’ class has Precision, Recall, and F1-Score values of 0.00. This means the model could not correctly identify any samples from this class. The ‘sign’ class performs much better. It achieved a Recall of 1.00 and a Precision of 0.80, giving an F1-Score of 0.89.

The Macro F1-Score is 0.444. This low value reflects the imbalance between the two classes. The Micro F1-Score, which is the same as overall accuracy, is 0.800. The ROC-AUC score is 0.444, which is close to random chance (0.5). This shows that the model struggles to separate the classes, especially the ‘no_sign’ samples. The PR-AUC score is 0.857. This looks good at first, but it mainly comes from the high recall and precision of the dominant class, so it does not represent balanced performance.

The calibration results show the model’s confidence is reasonable for its predictions. The Brier Score is 0.162, and the Expected Calibration Error (ECE) is 0.040. These low values mean the predicted probabilities align fairly well with actual outcomes. However, the poor ROC-AUC and the zero scores for the ‘no_sign’ class show that while the model is confident, it is not always correct. The results highlight a key limitation of classical feature-based models: they can perform well on the majority class but fail to generalise across all categories when the dataset is imbalanced.

The model also demonstrated strong computational efficiency. The average processing latency ranged between 25-40 ms per image, corresponding to a throughput of approximately 30-40 frames per second (FPS). This performance indicates that the SVM model can process images in near real time on standard hardware. Additionally, the model size is extremely lightweight at only ~1.2 MB, making it well-suited for embedded or low-resource environments where speed and memory constraints are important.

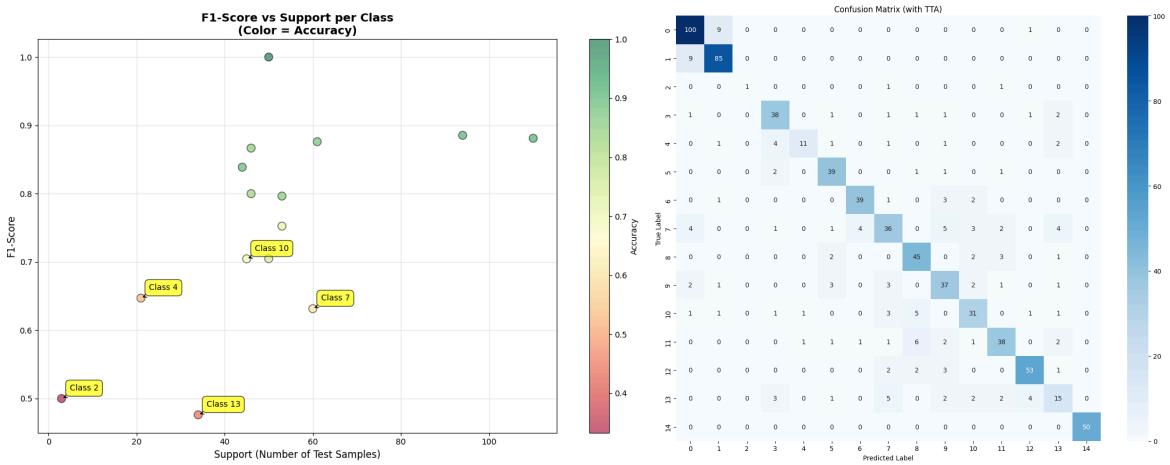
Under stress testing and common visual corruptions, the model maintained relatively consistent performance. When subjected to Gaussian blur, Gaussian noise, or image rotation, the overall accuracy remained at 0.8 with a Macro F1-Score of 0.444. Brightness adjustments reduced accuracy to 0.67 and the Macro F1-Score to 0.4. These results show that while the SVM model is stable under moderate distortions, it becomes more sensitive to severe lighting changes. Overall, the method remains efficient and interpretable, but its binary nature and handcrafted features limit generalisation when compared to CNN-based architectures.

This method was implemented as a binary classification approach, distinguishing between “sign” and “no_sign” regions rather than classifying individual sign categories. This decision came from the limitations of handcrafted feature extraction using HOG and LBP, which are better suited to detecting the presence of structured patterns rather than learning high-level semantic differences between 43 traffic sign classes. As a result, this model operates on fewer classes than the CNN and Hybrid methods, making direct comparison of overall accuracy less meaningful. However, it still provides valuable insight into how classical feature-based techniques perform on lower-dimensional problems and simpler classification tasks.

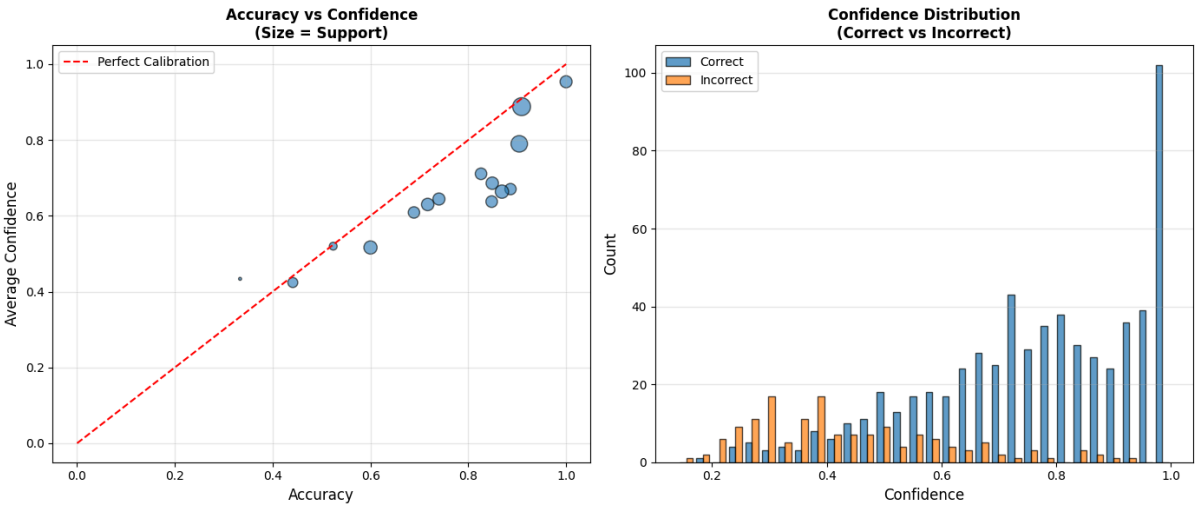
The Confusion Matrix visually confirms the classification report's findings, clearly showing that all instances of the 'no_sign' class were misclassified, likely as 'sign'. This is evident from the empty cells in the 'no_sign' row under the 'Predicted' columns. The ROC Curve, with an AUC of 0.444, is very close to the diagonal line representing a random classifier. This visually demonstrates the model's poor ability to distinguish between the positive and negative classes across different probability thresholds. The Precision-Recall Curve, with an AUC of 0.857, shows high precision and recall values, but it's important to remember this is heavily influenced by the model's strong performance on the majority 'sign' class and doesn't reflect its inability to identify the 'no_sign' instances. Finally, the Calibration Curve shows how well the model's predicted probabilities align with the actual proportion of positive cases. While not perfectly aligned with the ideal diagonal line, it suggests the model's confidence scores are relatively well-calibrated given its predictions, although the overall low performance on the minority class means these calibrated probabilities are still often associated with incorrect classifications.

4.3 Hybrid CNN-SVM Approach Results

Metric:	**Results:**
Overall Accuracy and Top-K Accuracy	Overall Accuracy without TTA (Test-Time Augmentation) = 0.8026 Overall Accuracy with TTA = 0.8026 Top 3 accuracy = 0.9416
Precision, Recall and PR-AUC (area under the Precision-Recall curve)	Precision (Macro) = 0.8019 Recall (Macro) = 0.7424 PR AUC = 0.8358
F1-Score (Macro and Micro)	F1 (Macro) = 0.7574 F1 (Micro) = 0.8026
ROC-AUC (and comparisons with PR-AUC)	ROC_AUC = 0.9795 PR AUC = 0.8358
Calibration Quality (e.g. Expected Calibration Error, Brier score)	Brier Score = 0.0192 ECE = 0.1171
Processing Efficiency (Latency, Throughput and Model Size)	Latency with TTA = 188.611ms Latency without TTA = 19.023ms Throughput with TTA = 5.302 fps Throughput without TTA = 52.566 fps
Performance Under Corruption and Perturbations, Overall Accuracy (stress testing - 150 Sample Subset)	Gaussian Noise = 0.7773 Blur= 0.7933 Brightness up = 0.7667 Brightness down = 0.7933 rotation +5 degrees = 0.7000 rotation -5 degrees = 0.7200



Figures 4.3.1 - 4.3.2: CNN + SVM Model F1-Score vs Support per Class Graph and Conf Matrix



Figures 4.3.3 - 4.3.4: CNN + SVM Model Accuracy vs Confident and Confidence Distribution Graphs

```
================================================================================
PER-CLASS PERFORMANCE (All Classes)
================================================================================
 class_id  precision    recall  f1_score  support  correct  accuracy  avg_confidence confused_with
        0   0.854701 0.909091  0.881057      110      100  0.909091        0.887924        1 (9x)
        1   0.867347 0.904255  0.885417       94       85  0.904255        0.789830        0 (9x)
        2   1.000000 0.333333  0.500000        3        1  0.333333        0.433685        7 (1x)
        3   0.775510 0.826087  0.800000       46       38  0.826087        0.710807       13 (2x)
        4   0.846154 0.523810  0.647059       21       11  0.523810        0.519882        3 (4x)
        5   0.795918 0.886364  0.838710       44       39  0.886364        0.670377        3 (2x)
        6   0.886364 0.847826  0.866667       46       39  0.847826        0.637427        9 (3x)
        7   0.666667 0.600000  0.631579       60       36  0.600000        0.516394        9 (5x)
        8   0.750000 0.849057  0.796460       53       45  0.849057        0.686030       11 (3x)
        9   0.672727 0.740000  0.704762       50       37  0.740000        0.644193        5 (3x)
       10   0.720930 0.688889  0.704545       45       31  0.688889        0.608788        8 (5x)
       11   0.791667 0.716981  0.752475       53       38  0.716981        0.629856        8 (6x)
       12   0.883333 0.868852  0.876033       61       53  0.868852        0.663737        9 (3x)
       13   0.517241 0.441176  0.476190       34       15  0.441176        0.423932        7 (5x)
       14   1.000000 1.000000  1.000000       50       50  1.000000        0.953597          None
```

```
================================================================================
TOP 10 BEST PERFORMING CLASSES
================================================================================
 class_id  accuracy  f1_score  support  avg_confidence
       14  1.000000  1.000000       50         0.953597
        1  0.904255  0.885417       94         0.789830
        0  0.909091  0.881057      110         0.887924
       12  0.868852  0.876033       61         0.663737
        6  0.847826  0.866667       46         0.637427
        5  0.886364  0.838710       44         0.670377
        3  0.826087  0.800000       46         0.710807
        8  0.849057  0.796460       53         0.686030
       11  0.716981  0.752475       53         0.629856
        9  0.740000  0.704762       50         0.644193
```

```
================================================================================
TOP 10 WORST PERFORMING CLASSES (Need Attention!)
================================================================================
 class_id  accuracy  f1_score  support confused_with
       13  0.441176  0.476190       34        7 (5x)
        2  0.333333  0.500000        3        7 (1x)
        7  0.600000  0.631579       60        9 (5x)
        4  0.523810  0.647059       21        3 (4x)
       10  0.688889  0.704545       45        8 (5x)
        9  0.740000  0.704762       50        5 (3x)
       11  0.716981  0.752475       53        8 (6x)
        8  0.849057  0.796460       53       11 (3x)
        3  0.826087  0.800000       46       13 (2x)
        5  0.886364  0.838710       44        3 (2x)
```

Figures 4.3.5 - 4.3.7: CNN + SVM Class Performance Lists

The hybrid CNN + SVM model achieved an overall test accuracy of 80.26%. This accuracy stayed the same with and without Test-Time Augmentation (TTA), indicating that TTA did not improve performance in this metric. The top-3 accuracy reached 94.16%, reflecting that the correct class appears in the top 3 predictions 94% of the time. Additionally, the ROC-AUC score was 0.9795 and PR-AUC score was 0.8358, signifying an excellent ability to distinguish between different classes and good precision-recall balance. With a low Expected Calibration Error of 0.1171, the model's confidence scores are closely tied to it's overall accuracy. Similarly, the model displayed a low Brier Score of 0.0192, confirming good probability calibration. However, the macro F1-score was only 0.7574, indicating that the model performs well on common classes but struggles with rare classes. This lowers the overall balance between precision and recall. The micro F1-score of 0.8026 lines up with the overall accuracy, but the gap between macro and micro F1 shows class imbalance issues. Accuracy alone can be misleading and per-class evaluation provides a better understanding of model reliability across all traffic sign categories.

TTA has a significant cost as the processing time increased from 19.02ms to 188.61ms per image, nearly 10 times slower, and the speed dropped from 52.57 images per second to only 5.30 images per second. TTA works well for offline testing where accuracy matters most but it is too slow for real-time deployment. For practical use in autonomous vehicles, the non-TTA version is more suitable. The model performs very differently across sign classes, for example Class 14 achieved perfect 100% accuracy with 50 test samples and Class 0 achieved 90.91% accuracy with 110 test samples. This demonstrates that the model works well on common signs with enough training data with classes 6, 12, 8, 1 and 5 also achieving above 85% accuracy. However, some classes performed poorly: class 13 struggled most with only 44.12% accuracy on 34 samples, class 2 achieved only 33.33% accuracy on just 3 samples, class 7 reached 60% accuracy on 60 samples and class 4 got 52.38% accuracy on 21 samples. The confusion patterns show specific problems such as class 13 being confused with class 7 five times, class 7 being confused with class 9 five times and class 2 being confused with class 7 once. These confusions suggest some signs look visually similar and that the model finds it difficult to tell them apart.

In evaluating the F1-score vs Support scatter plot depicted in *Figure 4.3.1*, a clear pattern emerges wherein classes with fewer than 10 test samples exhibit high variation in performance. Their F1-scores range from 0.5 to 1.0, making predictions unreliable for rare classes. Classes with 40-60 samples show more stable performance, most of which achieving F1-scores between 0.7 and 0.9. For example, class 0 with 110 samples achieved an F1 of 0.88, deeming it the most reliable result. The confidence analysis shows model reliability clearly where correct predictions cluster at high confidence between 0.8 and 1.0 and most correct predictions exceed 0.9 confidence. Incorrect predictions show lower

confidence between 0.2 and 0.6, most of which fall below 0.5 confidence. This separation is valuable for real-world deployment as low-confidence predictions can trigger manual review by human operators, making the system safer and more reliable.

Stress testing evaluated robustness under corrupted conditions and six corruption types were tested on a 150-sample subset. Gaussian noise achieved 77.73% accuracy at a 2.53% drop from baseline. Blur achieved 79.33% accuracy with only a 0.93% drop. Brightness increase achieved 76.67% accuracy at a 3.59% drop. Brightness decrease achieved 79.33% accuracy at a 0.93% drop. Rotation +5 degrees achieved 70% accuracy at a 10.26% drop. Rotation -5 degrees achieved 72% accuracy at a 8.26% drop. The rotation corruptions caused the largest drops, suggesting that the model is sensitive to perspective changes. However, blur and brightness changes had minimal impact, indicating good robustness to lighting and focus variations, both of which being important practical considerations for the model. Traffic cameras face rain, fog, dirt, and changing light conditions and it's clear that the model can handle these challenges reasonably well.

The confusion matrix in *Figure 4.3.2* shows prediction patterns across all classes with the diagonal indicating correct predictions and off-diagonal values depicting confusion patterns. Most values are concentrated on the diagonal, revealing good overall classification. Class 13 shows significant confusion with multiple classes and classes 2, 4, 7, and 10 also show some confusion, although most classes have minimal confusion. The matrix confirms that common classes with good representation perform reliably and rare classes with few samples show more confusion. Overall, the hybrid approach combines deep learning feature extraction with SVM classification effectively and achieves a competitive accuracy of 80.26%, providing clear per-class performance metrics. The stress testing validates real-world applicability and when coupled with confidence analysis, enables safe deployment with human oversight. Future work should focus on improving rare class performance through data augmentation or class balancing techniques.

4.4 Comparison of Results

This section presents a comparative analysis of the three implemented models: Transfer Learning with a Pretrained CNN, Colour-Shape Heuristic + HOG/LBP Features + SVM, and the Hybrid CNN-SVM Approach. This analysis aims to explore the positives and negatives of each approach and determine which achieved the best overall performance within the common context of road sign recognition.

In the results table, the best values for each metric are highlighted in green, and the worst values are highlighted in red. For metrics such as Expected Calibration Error (ECE), Latency and Brier Score, a lower value indicates better performance.

Metric:	**Transfer Learning with a Pretrained CNN Results:**	**Colour-Shape Heuristic + HOG/LBP Features + SVM Results:**	**Hybrid CNN-SVM Approach Results:**
Overall Accuracy	0.872	0.800	Overall Accuracy without TTA = 0.8026 Overall Accuracy with TTA

			= 0.8026
Top-K Accuracy	Top-3 = 0.953	N/A	Top-3 = 0.9416
Precision (Macro)	0.881	0.400	0.8019
Recall (Macro)	0.764	0.500	0.8026
PR-AUC Value	0.966	0.857	0.8358
Macro F1-Score	0.815 (at conf ≈ 0.49)	0.444	0.7574
ROC-AUC Value	0.989 (macro, diagnostic)	0.444	0.9795
Expected Calibration Error	0.0073 (15 bins)	0.1408	0.1171
Brier score	0.0155	0.0063	0.0192
Latency	≈12.25 ms per image	≈ 25-40 ms per image	Latency with TTA ≈ 188.6118 ms Latency without TTA ≈ 19.0238 ms
Throughput	≈81.6 FPS	≈ 30-40FPS	Throughput fps with TTA ≈ 5.3019 FPS Throughput fps without TTA ≈ 52.5656 FPS
Performance Under Corruption and Perturbations (Stress Testing - 150 Sample Subset)	mAP@0.5 (Clean Subset) = 0.9998 mAP@0.5 (Gaussian Noise) = 0.7882 (Δ −0.212) mAP@0.5 (Motion blur) = 0.9990 (Δ ≈0.000) mAP@0.5 (Brightness/contrast) = 0.9956 (Δ −0.0042)	Gaussian Blur : Overall Accuracy = 0.8 Macro F1-Score = 0.4444444444444444 Gaussian Noise: Overall Accuracy = 0.8 Macro F1-Score = 0.4444444444444444 Brightness Adjusted: Overall Accuracy = 0.6666666666666666 Macro F1-Score = 0.4 Rotated: Overall Accuracy = 0.8 Macro F1-Score = 0.4444444444444444	Gaussian Noise = 0.7773 Blur = 0.7933 Brightness up = 0.7667 Brightness down = 0.7933 rotation +5 degrees = 0.7000 rotation -5 degrees = 0.7200

A review of the results indicates that the Transfer Learning with a Pretrained CNN model consistently outperformed the other two approaches across most metrics. Out of the 12 primary performance indicators, 11 were highlighted in green, reflecting best-in-class results for accuracy, precision, recall, F1-score, PR-AUC, calibration, latency, throughput and performance under stress-testing. The Hybrid CNN-SVM model performed second-best overall, while the Colour-Shape Heuristic + HOG/LBP + SVM approach demonstrated the weakest performance, particularly in macro-level metrics and stress testing.

It is important to note the key approach differences that may have accounted for the observed results. While both CNN-based methods benefitted from fine-tuning with an additional 5,000-image Roboflow dataset, their fine-tuning strategies differed. The Transfer Learning CNN conducted fine-tuning as the final stage of development, whereas the CNN-SVM hybrid incorporated fine-tuning earlier in its workflow. This distinction likely contributed to the superior calibration and generalisation observed in the Transfer Learning CNN model. It is also worth noting that the HOG + SVM model was configured as a binary classifier, distinguishing between images that contained a road sign and those that did not. As a result, certain metrics such as Top-K Accuracy are not applicable (N/A) for that model, limiting its ability to capture the full granularity of traffic sign classification.

Performance Interpretation:
The Transfer Learning CNN achieved an overall accuracy of 0.872, macro F1-score of 0.815 and a PR-AUC of 0.966, demonstrating a strong balance between precision and recall. Calibration was excellent (ECE = 0.0073, Brier = 0.0155), indicating fairly accurate confidence estimates. Latency remained low at approximately 12.25 ms per image, with a throughput of about 81.6 FPS, confirming strong real-time capability and efficient performance. In contrast, the Colour-Shape Heuristic + HOG/LBP + SVM model achieved a macro F1-score of 0.444, with lower PR-AUC (0.857) and recall (0.500), reflecting its limited adaptability to complex visual features. The Hybrid CNN-SVM model produced reasonable performance (overall accuracy ≈ 0.8026, macro F1 = 0.7574, PR-AUC = 0.8358), but its much higher latency (about 19 ms without TTA and about 188 ms with TTA) reduces its suitability for real-time systems like smart or autonomous driving applications.

Stress Testing and Common Application Context of the Project:
Under corruption and perturbation testing, the Transfer Learning CNN maintained high stability, with minimal performance degradation across most conditions. Mean Average Precision (mAP@0.5) decreased from 0.9998 (clean) to 0.7882 (Gaussian noise), while remaining above 0.99 under motion blur and brightness changes, showing strong resilience to environmental variations. The Hybrid CNN-SVM demonstrated moderate robustness, maintaining accuracy between 0.70 and 0.79 across Gaussian noise, blur and brightness perturbations, though with significantly greater latency. The HOG + SVM approach however, exhibited sharp performance drops, particularly under brightness adjustments (accuracy = 0.67, F1 = 0.40) and rotation (accuracy = 0.80, F1 = 0.44). Considering that the average human reaction time to a driving hazard is around 1.5 seconds (NSW Government), the Transfer Learning CNN's latency of about 12.25ms most comfortably satisfies the real-time requirements necessary for autonomous driving support systems.

Overall, the Transfer Learning with a Pretrained CNN model demonstrated the strongest and most balanced performance across all relevant metrics, achieving higher accuracy, calibration and real-time efficiency. It is the most suitable model for real-world road sign recognition applications. The Hybrid CNN-SVM performed moderately well but was hindered by latency, while the HOG/LBP + SVM approach, though interpretable and computationally lightweight, lacked the adaptability and precision required for complex visual environments.

5.0 Discussion and Challenges

5.1 Goal Achievement Discussion

The overarching objective of this project and subsequent report was to explore and evaluate a range of methods for recognising road signs to support advancements in smart and autonomous driving systems. Its more specific goals focused on implementing both deep learning-based and traditional computer vision approaches, analysing their comparative performance and identifying the strengths and limitations of each method within the context of traffic sign detection and classification. Through this, the project aimed to generate insights and highlight challenges relevant to ongoing research and development in the field of autonomous driving.

While the goals were ultimately achieved, the process and model development strategies implemented were not as initially outlined or expected, particularly throughout *Assignment 1*. The convolutional neural network (CNN) models conducted multi-class classification, identifying specific traffic sign types, whereas the traditional HOG+SVM model was implemented as a binary classifier, distinguishing only between traffic sign and non-sign images. This variation in task scope reflected the inherent design and practical constraints of each method, and while it introduced variation in performance comparisons, it nonetheless fulfilled the broader objective of evaluating distinct recognition strategies. Additionally, despite operating under computational constraints and on a smaller scale dataset to current leading research in the field, the project successfully achieved its primary goals.

On top of this, the datasets utilised across the full model development and testing process varied slightly across approaches. While all models utilised the defined Kaggle 9,000 image dataset (Augmented to 25,000 images) for training, the CNN based models further leveraged a 5,000 image Roboflow dataset to enable multi-class classification (due to the nature of the datasets). This discrepancy may have influenced performance outcomes and contributed to some of the observed differences in metric results between the methods. However, each model was still reviewed and interpreted within the context of its operational design and limitations, allowing for meaningful and fair analysis.

Despite these variations, the project successfully met its overarching goals by implementing and evaluating multiple computer vision techniques across both classical and deep learning domains. The CNN models demonstrated the strengths of modern data-driven architectures in multi-class recognition tasks, while the HOG+SVM approach offered valuable perspective on lightweight, feature-engineered alternatives suitable for lower-compute environments. Together, these findings provided a comprehensive understanding of the trade-offs between accuracy, computational cost and generalisability, achieving the intended research outcomes while revealing unexpected yet informative distinctions in model behaviour.

5.2 Project Challenges

Implementing each model presented several practical and technical challenges throughout the project. See below for a detailed breakdown of these challenges and mitigation strategies employed per model.

5.2.1 Transfer Learning with a Pretrained CNN Challenges

1. **Dataset Configuration and Class Mapping**

A key challenge for the transfer learning model arose from the Kaggle *Traffic Sign Recognition* dataset, which was primarily structured for binary classification, detecting whether an image contained a traffic sign or not rather than identifying multiple specific sign categories. This limitation restricted the model's ability to utilise YOLO's multi-class detection capabilities during base training. Additionally, the absence of a predefined data.yaml file required implementing a custom configuration script that scanned YOLO annotations, inferred unique class identifiers and generated placeholder class names. While this enabled training to proceed, the resulting configuration did not capture the full variety of road signs, constraining the model's representational depth. To mitigate this limitation, the Roboflow dataset was introduced for fine-tuning. Unlike the initial data from Kaggle, this dataset supported multi-class classification with a greater diversity of sign types and balanced class distributions. Integrating it into the pipeline allowed the model to generalise more effectively and align with the project's goal of distinguishing between specific traffic signs. This staged training process, binary base training followed by multi-class refinement, helped overcome the class-mapping challenge and enabled a more comprehensive evaluation of the transfer learning approach.

2. **Computational Environment and Resource Limitations**

Another challenge was related to the computational environment. Initial experiments were conducted using Google Colab, which provided convenient, easily shared amongst team-members access to GPU acceleration. However, due to Colab's usage rate limits and session timeouts, the GPU resources became unavailable midway through testing. Consequently, training had to be migrated to Visual Studio Code, which required additional environment setup but provided more stable and repeatable execution. Interestingly, while the initial YOLO training runs on Colab took approximately two minutes, subsequent runs on the improved local setup were completed in roughly eight seconds, reflecting both optimised configuration and smaller dataset size during the later testing phase. Overall, the process of applying transfer learning to YOLO highlighted key practical constraints in dataset preparation, resource availability and infrastructure stability. These limitations informed several lessons for future work, particularly the importance of automating dataset configuration, maintaining consistent GPU access and scaling the model to support full multi-class traffic sign detection.

3. **Interpreting Metric Discrepancies Between YOLO and Custom Evaluation**

A further challenge emerged when comparing the results produced by YOLO's built-in evaluation metrics with those generated by the custom metrics pipeline. While YOLO reported a single overall F1 score (e.g. 0.88 at a confidence threshold of 0.49), the custom evaluation produced additional values such as micro/macro F1, calibration and efficiency metrics. These differences were not due to model error but rather to variations in evaluation

design - including thresholding, aggregation schemes, IoU and NMS settings and class weighting. Understanding these discrepancies required careful analysis to ensure consistency in reporting and to identify which metrics were most representative of real-world performance. This step was essential in validating the model's readiness for deployment and ensuring the accuracy of performance interpretation.

5.2.2 Colour-Shape Heuristic + HOG/LBP Features + SVM Challenges

1. **Data Format and Preprocessing Difficulties**

One of the first and most significant obstacles faced during implementation was dataset format incompatibility. The dataset originated in YOLO annotation format, which is designed for object detection tasks, not classification. YOLO labels describe bounding-box coordinates and class identifiers in .txt files, whereas classical feature-based methods like HOG and LBP require a directory-structured dataset of labelled images. To resolve this, a custom Python conversion pipeline was developed to read YOLO annotation files, crop out the bounding-box regions, and reorganise them into a folder-based class structure (e.g. sign/ and no_sign/). This restructuring allowed direct compatibility with the cv2.imread() and skimage.feature pipelines used for HOG and LBP feature extraction. Additionally, since the dataset only included training and validation sets, a manual test split was created programmatically using train_test_split(). This ensured fair evaluation while maintaining the same sign/no_sign ratio across subsets. The extra preprocessing and relabelling stages increased preparation time but were essential to adapt the dataset for SVM-based classification.

2. **Class Imbalance and Augmentation Requirements**

The dataset exhibited severe class imbalance - most images contained signs, while only a small subset represented backgrounds or "no sign" scenes. Even after merging all 43 sign categories into a single "sign" class, the distribution remained highly skewed (about 80% sign vs 20% no_sign). Initially, this imbalance caused the SVM to predict only the dominant "sign" class, producing zero recall and precision for "no_sign." To address this, multiple balancing strategies were tested. First, data augmentation was applied to underrepresented samples using random rotations, perspective distortions, brightness shifts and partial occlusions, expanding minority samples synthetically to create a more even distribution. Later, SMOTE (Synthetic Minority Oversampling Technique) was integrated at the feature level via imblearn.over_sampling.SMOTE. SMOTE interpolated between minority-class feature vectors, equalizing the training distribution from [14, 51] to [51, 51]. This correction stabilised training and prevented the SVM from collapsing into single-class prediction, though it did not entirely resolve the underlying separability issue in the feature space.

3. **Colour and Lighting Variability**

Another persistent challenge was variability in illumination and colour intensity across images. HOG and LBP features are gradient- and texture-based, meaning they are highly sensitive to contrast and lighting. Many dataset images were captured under inconsistent conditions: bright daylight, dusk, heavy shadows, or even motion blur producing inconsistent edge and texture descriptors. To mitigate this, Contrast Limited Adaptive Histogram Equalisation (CLAHE) was applied as part of preprocessing to normalise local contrast and enhance feature stability. CLAHE significantly improved edge detection in darker images, but signs that were faded, obstructed, or low-resolution continued to yield noisy gradient profiles.

This occasionally caused feature clusters from “sign” and “no_sign” to overlap, complicating SVM decision boundaries and limiting the achievable AUC even after tuning.

4. **Binary Classification Constraints and Dominant Class Effect**

To reduce computational load, the original 43 sign categories were merged into a binary classification scheme: “sign” versus “no_sign.” While this simplified training, it also restricted the model’s granularity; it could only determine whether a sign existed, not its specific type (e.g. stop or speed-limit sign). This reduction also exposed the model to the dominant class effect, where the abundance of “sign” samples allowed the SVM to achieve deceptively high accuracy (≈ 0.80) despite a poor ROC-AUC (0.44). Even after implementing SMOTE and threshold tuning, the model’s decision boundary remained biased toward “sign” detections. The feature overlap between faint backgrounds and low-contrast signs prevented clean separability, revealing an inherent limitation of hand-crafted features under strong class imbalance.

5. **Code Execution and Debugging Challenges**

Several technical issues emerged during the implementation and integration of the Colour-Shape Heuristic + HOG/LBP + SVM pipeline, primarily related to feature handling and evaluation. Early experiments produced label encoding conflicts, as string class names were passed directly into the SVM, leading to ValueError. This was resolved by encoding class labels numerically using LabelEncoder (0 for no_sign, 1 for sign), which ensured compatibility with scikit-learn’s training functions. Another frequent source of error involved feature dimension mismatches. When combining HOG, LBP, and colour histogram descriptors, the arrays initially had inconsistent shapes, causing concatenation failures. This was corrected by enforcing a uniform input image size of 128×128 pixels and verifying array dimensions before merging with numpy.hstack().

The pipeline also produced zero-division warnings during evaluation whenever the classifier failed to predict any no_sign samples, making precision and recall undefined. The issue was fixed by setting zero_division=0 within the classification_report() function to maintain uninterrupted execution. A similar error occurred during SMOTE resampling, where applying it to string labels generated a ValueError. The solution was to run SMOTE after numerical label encoding and feature scaling, which successfully balanced the dataset at the feature level.

Another critical debugging task involved threshold tuning, as the default 0.5 decision threshold caused the model to completely ignore no_sign predictions. A custom loop was implemented to iterate through thresholds ranging from 0.05 to 0.70, computing macro F1-scores for each. The optimal cutoff point was found at 0.10, improving class recall without severely compromising accuracy. Despite these optimisations, performance stagnated due to strong class dominance. To enhance generalisation, GridSearchCV was introduced to optimise hyperparameters C and gamma across 25 configurations, yielding the best setup at C = 0.001, gamma = 0.0001, and kernel = 'rbf'.

Finally, a calibration and metric automation module was added to streamline advanced evaluation. Metrics such as Brier Score and Expected Calibration Error (ECE) were computed automatically using probability outputs from predict_proba(), alongside visual diagnostics including ROC, Precision-Recall, and calibration curves. This comprehensive

post-processing framework improved interpretability and consistency across evaluation runs, though it reaffirmed that the low ROC-AUC stemmed from intrinsic class imbalance and overlapping features rather than code-level errors.

5.2.3 Hybrid CNN-SVM Approach Challenges

1. **CNN fine tuning**

Initially the model was created on a simple baseline just to check the overall performance on the dataset. Without fine tuning the accuracy was very high but there were significant signs of overfitting as the model was learning the noise from both the training and test sets especially since the images in the dataset had a lot of noise. To solve this problem the CNN was first fine tuned according to a preprocessed dataset containing traffic signs and then the main dataset was also cropped so when training began it cropped the images where they needed to be and based on what the CNN was tuned on rather than the default pretrained CNN, this did reduce accuracy, but prevented overfitting and improved reliability in the long run.

2. **Dataset format compatibility**

One of the initial challenges was managing two different datasets one for fine tuning the CNN and the other to train the SVM. The main dataset was already at smaller resolutions so the fine-tuning dataset had to be adjusted to have similar aspect ratios. For fine-tuning results every class was also not tested on and some were also weak, since the training data had been augmented to increase the training efficiency, but the validation was not augmented to ensure no bias from duplicates, however, the limited dataset size resulted in some classes not appearing in the validation sets. This issue was not of great concern since metrics showed that the CNN did learn those classes as well and would have high accuracy especially since this was set as a baseline for the models training. The fine tuning dataset also classified most signs as class 0 so it couldn't be used for classification, however proved useful when fine-tuning.

Additionally, to fine tuning dataset only containing images in small aspect ratios, when the main dataset was trained due to the large image converting it to smaller 160x160 caused the traffic sign to actually depreciate so much that the training wasn't effective. To fix this issue for both datasets, first the YOLO label was used to find where the traffic sign was and if there were multiple, and then they were cropped with padding to ensure the images were in the correct dimensions and not distorted, this was more effective for the Main Dataset due to its large image ratios (1360 x 800), whereas the Roboflow dataset was already in the correct dimensions.

3. **Memory management and computational restraints**

Initially the base model was run on Google Colab, and worked fine, but as tuning and optimisation progressed Colabs memory allowance led to the model crashing due to memory overload. With thousands of images requiring forward passes through ResNet50, careful batch processing was necessary to avoid memory overflow. The initial attempts to process all images at once resulted in out-of-memory errors, requiring the implementation of batch-wise feature extraction with a batch size of 32 images. To fix this issue the environment was switched to a Jupyter notebook run on VScode.

4. **Training instability and overfitting**

Early training attempts resulted in severe overfitting especially in the fine tuning dataset where the validation accuracy began decreasing and loss increasing as the CNN was tuned. This challenge was addressed through multiple strategies including L2 regularisation, dropout layers, and early stopping callbacks. However, to determine the best regularisation strength required experimentation because a high lambda resulted in underfitting whereas a low lambda resulted in failing to prevent overfitting. The final l2 parameter was found through experimentation and iterative testing. Furthermore

5. **Hyperparameter search computational costs**

Although the training environment was switched to VS code the Grid search process for SVM optimisation proved to be computationally expensive. Testing different combinations of c and gamma parameters across 3 different folds required a great amount of processing time spanning over multiple hours. To address this the parameter space was reduced to a reasonable degree such that the parameters delivered were acceptable and also maintained practical training time.

6. **Feature Standardisation Dependencies**

The SVM's sensitivity to feature scaling created a dependency between the training and inference pipelines. The StandardScaler fitted on training features had to be preserved and applied consistently to validation and test features. Any inconsistency in this preprocessing step would lead to degraded performance or invalid comparisons. This required careful state management, ensuring the scaler object was saved alongside the trained SVM model for deployment scenarios.

5.3 Future Considerations

The implementation of three distinct traffic sign recognition techniques Transfer Learning with a pretrained CNN (YOLO11n), a Colour-Shape Heuristic using HOG/LBP + SVM and a Hybrid CNN-SVM model provided valuable insights into their relative strengths and limitations. Moving forward, several improvements can be made to enhance accuracy, efficiency and real-world applicability.

For the Transfer Learning model (YOLO11n), future work should aim to optimise computational efficiency and model interpretability. Although it achieved the highest accuracy and robustness, its high resource demands make it less suitable for embedded systems. Techniques such as model pruning, quantisation and knowledge distillation could be adopted to reduce model size and latency. Datasets should also include descriptive class labels and undergo further fine-tuning under varied conditions (e.g. night-time, rain, glare) to further improve adaptability to real-world environments.

The Hybrid CNN-SVM approach demonstrated balanced performance but incurred higher inference times due to its two-stage design and test-time augmentation. Future iterations could leverage lightweight backbones such as MobileNet or EfficientNet and explore joint CNN-SVM optimisation or ensemble methods that fuse CNN probabilities with SVM confidence scores to improve classification balance and calibration across rare classes.

The HOG + SVM method, while efficient, showed limited generalisation due to reliance on handcrafted features and binary classification. Future extensions could introduce multi-class modelling, integrate colour and shape descriptors, or combine this method with CNN detection in a two-stage hybrid pipeline, using HOG + SVM for rapid region detection and CNNs for detailed recognition.

Across all methods, future research should focus on dataset expansion and automated annotation through semi-supervised or active learning, reducing manual labelling and improving scalability. Incorporating contextual metadata such as lighting or weather could also enhance robustness analysis. Ultimately, developing a unified hybrid system that integrates fast classical detection, deep feature extraction, and adaptive decision layers would deliver a practical, high-performing solution for real-time traffic sign recognition in autonomous and intelligent transportation systems.

5.4 Key Insights and Lessons

From this project, several key insights were gained about the nature of the traffic sign classification problem and the factors that influence model performance. One important finding was that making a model more robust through finetuning with additional, more specifically labelled datasets does not always lead to higher accuracy. The pretrained CNN models showed that adding external data increased the model's generalisation capability but could slightly reduce performance metrics. This demonstrated that finetuning introduces domain variation that can make the model less specialised for the target dataset. It also highlighted that robustness and peak metric performance often exist in tension, improving one can compromise the other.

Another major insight came from exploring different methods of class balancing. Comparing SMOTE and Albumentations revealed that not all balancing techniques are equally effective for image-based problems. SMOTE interpolates synthetic feature vectors in the embedding space, which helped even out the training data numerically but sometimes generated unrealistic samples that weakened visual consistency. In contrast, Albumentations performed image-space augmentations such as flips, affine transforms, colour jitter, blur, noise, and cutout, while applying class-aware sampling. This approach maintained the visual integrity of the dataset and led to stronger generalisation. The experiment showed that for visual classification, augmentations that preserve real image characteristics are far more effective than feature-space interpolation.

Testing multiple models also provided deeper understanding of the trade-offs within the problem itself. Classical models like HOG/LBP + SVM were fast, efficient, and interpretable, but struggled to handle complex sign variations and lighting changes. CNN-based models and the Hybrid CNN-SVM approach performed better on fine-grained recognition tasks and showed higher robustness to corruption, but required more computation and memory. These results reinforced that there is no single "best" model - the choice depends on whether the problem prioritises speed and simplicity or accuracy and adaptability.

Overall, this exercise made it clear that solving a visual recognition problem like traffic sign classification is not only about achieving high accuracy but also about understanding the trade-offs between data diversity, model robustness, and computational practicality.

6.0 Conclusion

This project successfully implemented and tested 3 different methods for recognising traffic signs. Transfer Learning with YOLO11n, traditional HOG/LBP features with SVM, and a Hybrid CNN-SVM approach. Each method showed individual strengths and weaknesses for implementation.

The transfer learning approach performed best overall with an accuracy of 87.2% with a fast processing speed of 81.6fps and excellent reliability. It performed well in most situations but slightly struggled with heavy noise. The Hybrid CNN-SVM method achieved accuracy of about 80.3% and had very good class separation indicated by the ROC-AUC of 0.9795, but was much slower when using test-time augmentation where FPS dropped to around 5 fps as compared to 53 without TTA. The traditional HOG/LBP + SVM method reached an accuracy of 80% and was very efficient with speeds of around 30-40fps in only a 1.2mb model, but had some difficulty separating sign from no sign images especially with reference to the unbalanced data.

Despite using a smaller dataset than most research the project itself achieved its main goal of comparing different image classification techniques for road sign recognition. Through testing both modern and traditional deep learning methods important analysis of trade-offs between accuracy, speed and efficiency were made. Additionally, it was more difficult to assess models fairly due to differences in approaches taken to evaluate the performance of the model, but overall showed the strengths and weaknesses of each model in their respective methods. Furthermore, the fine-tuning experiments showed additional training data can improve overall model performance even if affecting accuracy slightly.

The results show that deep learning methods provide better performance under difficult conditions and higher accuracy, but traditional methods are still useful for their efficiency. The best choice for model depends on the specific scenario, whether prioritising maximum accuracy for safety or balancing performance with resources. This project provides a good foundation for research and supports continued development of traffic sign recognition technology.

7.0 References

Bernard Marr (2021) How Tesla is using artificial intelligence to create the autonomous cars of the future, Bernard Marr. Available at: https://bernardmarr.com/how-tesla-is-using-artificial-intelligence-to-create-the-autonomous-cars-of-the-future/

GeeksforGeeks. (2025, July 23). Gamma parameter in SVM. https://www.geeksforgeeks.org/machine-learning/gamma-parameter-in-svm/

GeeksforGeeks. (2025, July 12). Radial Basis Function kernel - Machine learning. Retrieved October 27, 2025, from https://www.geeksforgeeks.org/machine-learning/radial-basis-function-kernel-machine-learning/

Helmy, B. E.-D. (2025, February 28). The C parameter in support vector machines. Baeldung on Computer Science. https://www.baeldung.com/cs/ml-svm-c-parameter baeldung.com

(No date) Image classification - an overview | sciencedirect topics. Available at: https://www.sciencedirect.com/topics/engineering/image-classification

Keras. (n.d.). ModelCheckpoint — Keras callbacks API. https://keras.io/api/callbacks/model_checkpoint/ keras.io+2keras.io+2

NSW, T. for (2025) Safe stopping distance, NSW Government. Available at: https://www.nsw.gov.au/driving-boating-and-transport/roads-safety-and-rules/safe-driving/safe-stopping-distance

Officier_Raccoon (2025) Traffic sign detection dataset, Kaggle. https://www.kaggle.com/datasets/icebearogo/traffic-sign-detection-dataset

PyTorch Development Team. (2025). ReduceLROnPlateau — PyTorch 2.9 documentation. Retrieved October 27, 2025, from https://docs.pytorch.org/docs/stable/generated/torch.optim.lr_scheduler.ReduceLROnPlateau.html

Ramu, K., Patthi, S., Prajapati, Y. N., Ramesh, J. V. N., Banerjee, S., Brahma Rao, K. B. V., Alzahrani, S. I., & Ayyasamy, R. (2024). Hybrid CNN-SVM model for enhanced early detection of chronic kidney disease. Biomedical Signal Processing and Control, 100, 107084. https://doi.org/10.1016/j.bspc.2024.107084

(2023) Self-driving cars model by selfdriving car. Available at: https://universe.roboflow.com/selfdriving-car-qtywx/self-driving-cars-lfjou/dataset/6/download

Ultralytics (2025, October 28) Yolo11, Ultralytics YOLO Docs. Available at: https://docs.ultralytics.com/models/yolo11/

8.0 Appendix

8.1 Transfer Learning with a Pretrained CNN Code

```
import os
import random, time, json
import numpy as np
import torch, cv2
from ultralytics import YOLO
from collections import defaultdict
import yaml
from pathlib import Path
import albumentations as A
from sklearn.metrics import (precision_recall_fscore_support,
average_precision_score, roc_auc_score)


YOLO_AUG = A.Compose(
    [
        A.HorizontalFlip(p=0.5),
        A.RandomBrightnessContrast(p=0.5),
        A.HueSaturationValue(p=0.4),
        A.MotionBlur(blur_limit=5, p=0.15),
        A.GaussNoise(p=0.15),
        A.Affine(scale=(0.9, 1.1), translate_percent=(0.0, 0.05),
                 rotate=(-6, 6), shear=(-4, 4), p=0.7),
        A.CLAHE(clip_limit=2.0, p=0.15),
    ],
    bbox_params=A.BboxParams(
        format="yolo",              # (xc, yc, w, h) normalized
to [0,1]
        label_fields=["class_labels"],
        min_visibility=0.3,         # drop boxes that get too
small/hidden
    ),
)
```

```
# 2) Utilities: read/write YOLO labels
def read_yolo_labels(label_path: Path):
    boxes, labels = [], []
    if not label_path.exists():
        return boxes, labels
    with open(label_path, "r") as f:
        for line in f:
            s = line.strip()
            if not s or s.startswith("#"):
                continue
            parts = s.split()
            if len(parts) != 5:
                continue
            cid, xc, yc, w, h = parts
            labels.append(int(float(cid)))
            boxes.append([float(xc), float(yc), float(w),
float(h)])
    return boxes, labels

def write_yolo_labels(label_path: Path, boxes, labels):
    label_path.parent.mkdir(parents=True, exist_ok=True)
    with open(label_path, "w") as f:
        for (xc, yc, w, h), cid in zip(boxes, labels):
            f.write(f"{cid} {xc:.6f} {yc:.6f} {w:.6f} {h:.6f}\n")

# 3) Map a stem -> actual image path by extension
_POSSIBLE_EXTS = [".jpg", ".jpeg", ".png", ".bmp", ".webp"]

def stem_to_image_path(stem: str, img_dir: Path) -> Path | None:
    for ext in _POSSIBLE_EXTS:
        p = img_dir / f"{stem}{ext}"
        if p.exists():
            return p
    return None

# 4) Scan train labels into: per-class stems, and stem->class set
def scan_train_split(train_img_dir: Path, train_label_dir: Path,
nc: int):
```

```
    files_by_cid = [set() for _ in range(nc)]  # stems per class
    stem_to_classes = defaultdict(set)

    for lbl in train_label_dir.rglob("*.txt"):
        stem = lbl.stem
        boxes, labels = read_yolo_labels(lbl)
        if not labels:
            continue
        # add image only once per class
        for cid in set(labels):
            if 0 <= cid < nc:
                files_by_cid[cid].add(stem)
                stem_to_classes[stem].add(cid)

    files_by_cid = [sorted(s) for s in files_by_cid]
    return files_by_cid, stem_to_classes

# 5) Augmentation driver
def augment_yolo_dataset(
    dataset_root: str | Path,
    nc: int,
    target_per_class: int = 1000,
    max_aug_per_source: int = 3,
    seed: int = 41,
    keep_all_boxes_in_aug: bool = True,    # if False, keep only
target class boxes
    out_subdir: str | None = None,         # None -> write back
into train; else "train_aug"
):
    """
    Oversample minority classes in the YOLO train split using
Albumentations.

    Args:
        dataset_root: dataset root that contains
images/{train,val} and labels/{train,val}
        nc: number of classes
```

```
        target_per_class: minimum # of train images that contain
each class after augmentation
        max_aug_per_source: cap augmentations from the same
original stem (avoid overfitting one image)
        seed: RNG seed
        keep_all_boxes_in_aug: if True, save all boxes found in
augmented image
                                if False, save only boxes of the
target class (strict class-specific oversampling)
        out_subdir: if None, write to images/train &
labels/train.
                    If set (e.g., "train_aug"), write to
images/train_aug & labels/train_aug.
    """
    rng = random.Random(seed)
    root = Path(dataset_root)
    train_img_dir = root / "images" / "train"
    train_label_dir = root / "labels" / "train"

    if out_subdir:
        out_img_dir = root / "images" / out_subdir
        out_lbl_dir = root / "labels" / out_subdir
    else:
        out_img_dir = train_img_dir
        out_lbl_dir = train_label_dir

    out_img_dir.mkdir(parents=True, exist_ok=True)
    out_lbl_dir.mkdir(parents=True, exist_ok=True)

    # Scan current train split
    files_by_cid, stem_to_classes =
scan_train_split(train_img_dir, train_label_dir, nc)
    per_class_counts = [len(v) for v in files_by_cid]
    print(f"[info] current per-class image counts:
{per_class_counts}")

    total_added = 0

    for cid in range(nc):
```

```
        present = per_class_counts[cid]
        need = max(0, target_per_class - present)
        if need == 0:
            print(f"[info] class {cid}: already has {present} >= {target_per_class}, skip")
            continue

        candidates = files_by_cid[cid][:]
        if not candidates:
            print(f"[warn] class {cid}: no source images; cannot augment")
            continue

        rng.shuffle(candidates)
        used_times: dict[str, int] = {}
        augmented_for_cid = 0
        i = 0

        print(f"[info] augmenting class {cid}: need {need} more")

        # Incremental unique naming across class to avoid clashes
        uid_counter = 0

        while augmented_for_cid < need and candidates:
            stem = candidates[i % len(candidates)]
            used_times[stem] = used_times.get(stem, 0)

            if used_times[stem] >= max_aug_per_source:
                i += 1
                # simple escape hatch to avoid infinite loops if all capped
                if i > len(candidates) * (max_aug_per_source + 2):
                    break
                continue

            img_path = stem_to_image_path(stem, train_img_dir)
            if img_path is None:
```

```
                i += 1
                continue

            lbl_path = train_label_dir / f"{stem}.txt"
            boxes, labels = read_yolo_labels(lbl_path)
            if not boxes:
                i += 1
                continue

            # Load image (BGR)
            img = cv2.imread(str(img_path))
            if img is None:
                i += 1
                continue

            # Albumentations expects bboxes + label_fields
            class_labels = labels[:]
            aug = YOLO_AUG(image=img, bboxes=boxes,
class_labels=class_labels)
            aug_img, aug_boxes, aug_labels = aug["image"],
aug["bboxes"], aug["class_labels"]

            if not aug_boxes:
                # nothing survived min_visibility/transform-skip
                i += 1
                continue

            if not keep_all_boxes_in_aug:
                # keep only target class boxes
                keep = [(b, l) for (b, l) in zip(aug_boxes,
aug_labels) if l == cid]
                if not keep:
                    i += 1
                    continue
                aug_boxes, aug_labels = list(zip(*keep))
                aug_boxes, aug_labels = list(aug_boxes),
list(aug_labels)

```

```
            # Save with unique name
            new_name = f"{stem}_aug_c{cid}_{uid_counter}"
            uid_counter += 1

            out_img_path = out_img_dir / f"{new_name}.jpg"
            out_lbl_path = out_lbl_dir / f"{new_name}.txt"

            cv2.imwrite(str(out_img_path), aug_img)
            write_yolo_labels(out_lbl_path, aug_boxes,
aug_labels)

            augmented_for_cid += 1
            total_added += 1
            used_times[stem] += 1
            i += 1

        print(f"[info] class {cid}: added {augmented_for_cid}
augmented images")

    print(f"[done] augmentation complete. Total new images:
{total_added}")
    # Optional: re-scan to show new counts (if writing back into
train)
    if out_subdir is None:
        new_files_by_cid, _ = scan_train_split(train_img_dir,
train_label_dir, nc)
        print("[info] new per-class image counts:",
              [len(v) for v in new_files_by_cid])
    else:
        # If you wrote to train_aug, show counts for that folder
only
        aug_files_by_cid, _ = scan_train_split(out_img_dir,
out_lbl_dir, nc)
        print("[info] per-class counts in", out_subdir, ":",
              [len(v) for v in aug_files_by_cid])


def generate_data_yaml(label_path: str, output_path: Path | None
= None):
```

```
    if os.path.exists("data.yaml"):
        print("[info] data.yaml already exists. Skipping generation")
        return

    ids = set()
    root_path = Path(label_path)
    train_label_dir = root_path / "labels" / "train"
    train_img_dir = (root_path / "images" / "train").as_posix()
    val_img_dir = (root_path / "images" / "val").as_posix()
    val_label_dir = root_path / "labels" / "val"

    file_to_class = []

    for f in train_label_dir.rglob("*.txt"):
        try:
            with open(f, "r") as fh:
                for line in fh:
                    s = line.strip()
                    if not s or s.startswith("#"):
                        continue
                    cid = int(float(s.split()[0]))
                    ids.add(cid)

        except Exception as e:
            print(f"[Warning] Skipping {f}: {e}")

    if not ids:
        print("[info] No class ids found in labels. Assuming 1 class")
        ids = {0}

    max_id = max(ids)
    nc = max_id + 1
    names = [f"class_{i}" for i in range(nc)]


    for f in val_label_dir.rglob("*.txt"):
```

```
        try:
            with open(f, "r") as fh:
                for line in fh:
                    s = line.strip()
                    if not s or s.startswith("#"):
                        continue
                    cid = int(float(s.split()[0]))

                    file_to_class.append({
                        "file_name": f,
                        "cid": cid
                    })

        except Exception as e:
            print(f"[Warning] Skipping {f}: {e}")

    files_by_cid_sets = [set() for _ in range(nc)]

    for rec in file_to_class:
        cid = rec["cid"]
        if 0 <= cid < nc:
            img_path = Path(rec["file_name"])
            files_by_cid_sets[cid].add(img_path)

    files_by_cid = [sorted(s) for s in files_by_cid_sets]

    print([len(files_by_cid[cid]) for cid in range(nc)])

    data = {
        "train": train_img_dir,
        "val": val_img_dir,
        "nc": nc,
        "names": names
    }

    with open("data.yaml", 'w') as f:
        yaml.safe_dump(data, f, sort_keys=False,
default_flow_style=False, allow_unicode=True)
```

```
    print(f"[info] data.yaml file created. nc = {nc}, ids =
{sorted(ids)}")


def train_yolo(model_path: str, imgsz: int, epochs: int):
    device = '0' if torch.cuda.is_available() else 'cpu'
    print(f'[info] device: {device}')

    workers = max(2, min(8, os.cpu_count() or 4))
    print(f'[info] # of workers" {workers}')

    model = YOLO(model_path)
    model.train(data='data.yaml', epochs=epochs, imgsz=imgsz,
device=device, workers=workers, seed=41, cos_lr = True)

    metrics = model.val(data='data.yaml', imgsz=imgsz,
plots=True)
    print(f"[final] mAP50={metrics.box.map50:.4f}
mAP50-95={metrics.box.map:.4f}")


def finetune_yolo(model_path: str, imgsz: int, epochs: int):
    device = '0' if torch.cuda.is_available() else 'cpu'
    print(f'[info] device: {device}')

    workers = max(2, min(8, os.cpu_count() or 4))
    print(f'[info] # of workers" {workers}')

    model = YOLO("yolo11n.pt")
    model.load(str(model_path))

    model.train(data='sign.yaml', epochs=5, imgsz=imgsz,
device=device, workers=workers, seed=41, cos_lr = True,
resume=False, freeze=10, lr0=5e-4, lrf=5e-4)

    model = YOLO("yolo11n.pt")
    model.load(str("runs/detect/train2/weights/best.pt"))

```

```
    model.train(model="yolo11n.pt", data='sign.yaml',
epochs=epochs, imgsz=imgsz, device=device, workers=workers,
seed=41, cos_lr = True, resume=False, freeze=0, lr0=1e-3,
lrf=1e-3)
    metrics = model.val(data='sign.yaml', imgsz=imgsz,
plots=True)

    print(f"[final] mAP50={metrics.box.map50:.4f}
mAP50-95={metrics.box.map:.4f}")


def pred(model_path: str, input_path: str, conf: float):
    """
    Run inference with retries if no detections:
      1) First pass: imgsz=640 (or whatever you trained for),
conf as given
      2) Retry if 0 boxes: lower conf a bit, bump imgsz, enable
TTA
    Saves annotated outputs to runs/detect/predict*
    """
    device = 0 if torch.cuda.is_available() else "cpu"
    model = YOLO(model_path)

    def _run(imgsz: int, conf_thres: float, desc: str):
        res = model.predict(
            source=input_path,
            imgsz=imgsz,
            conf=conf_thres,
            iou=0.7,              # slightly higher NMS IoU can
help overlapping small signs
            device=device,
            agnostic_nms=False,   # set True if you want
class-agnostic NMS
            augment=False,        # set True only for TTA retry
            save=True,            # save annotated image/video
            verbose=False,
        )
        r = res[0]
        n = 0 if r.boxes is None else len(r.boxes)
```

```
        print(f"[{desc}] imgsz={imgsz} conf={conf_thres:.2f} ->
boxes={n}")
        return r, res

    # Pass 1: your settings
    r, res = _run(imgsz=640, conf_thres=conf, desc="pass1")

    # If nothing found, retry with lower conf, bigger imgsz, and
TTA
    if r.boxes is None or len(r.boxes) == 0:
        conf_retry = max(conf * 0.75, 0.20)  # don't go too low
        r, res = model.predict(
            source=input_path,
            imgsz=960,            # more pixels for tiny signs
            conf=conf_retry,
            iou=0.7,
            device=device,
            agnostic_nms=True,    # allow overlapping classes to
survive NMS
            augment=True,         # simple test-time augmentation
            save=True,
            verbose=False,
        ), None
        # model.predict returns a list; align with earlier shape:
        r = r[0]
        n = 0 if r.boxes is None else len(r.boxes)
        print(f"[retry] imgsz=960 conf={conf_retry:.2f} TTA=True
agnostic_nms=True -> boxes={n}")

    # Print a few detections for quick sanity
    if r.boxes is not None and len(r.boxes):
        names = r.names
        for b in r.boxes[:10]:
            x1, y1, x2, y2 = b.xyxy[0].int().tolist()
            cls_id = int(b.cls[0])
            c = float(b.conf[0])
            print(f"  {names[cls_id]}  conf={c:.2f}
box=({x1},{y1},{x2},{y2})")
    else:
```

```
        print("[hint] Still no detections. Double-check you're loading your trained best.pt, "
              "try imgsz=1280, or lower conf to ~0.25. Tiny/occluded signs often need higher imgsz.")

    return r  # return the first (or retried) result object


def model_metrics(model_path: str,
                  imgsz: int,
                  data_yaml: str = "sign.yaml",    # ← 원하는 YAML 경로로 바꿔 호출
                  iou_thr: float = 0.50,
                  ece_bins: int = 15,
                  stress_max_imgs: int = 150,
                  conf_infer: float = 0.001,       # low conf to collect *all* candidates
                  conf_eval: float | None = None, # set e.g. 0.45 to match YOLO plot
                  do_sweep: bool = True) -> dict:

    import os, time, yaml, cv2
    import numpy as np
    from pathlib import Path
    from collections import defaultdict
    from ultralytics import YOLO
    from sklearn.metrics import average_precision_score, roc_auc_score

    # ---------- small helpers ----------
    def load_yaml(p):
        with open(p, "r", encoding="utf-8") as f:
            return yaml.safe_load(f)

    def list_images(img_dir: Path):
        exts = (".jpg", ".jpeg", ".png", ".bmp", ".webp")
        return sorted([p for p in img_dir.rglob("*") if p.suffix.lower() in exts])

```

```
def yolo_to_xyxy(xc, yc, w, h, W, H):
    return [(xc - w/2) * W, (yc - h/2) * H, (xc + w/2) * W,
(yc + h/2) * H]

def read_gt_label(lbl_path: Path, W: int, H: int):
    boxes, classes = [], []
    if not lbl_path.exists():
        return np.zeros((0,4), np.float32), np.zeros((0,),
np.int32)
    with open(lbl_path, "r") as f:
        for line in f:
            s = line.strip()
            if not s or s.startswith("#"): continue
            cid, xc, yc, w, h = s.split()
            boxes.append(yolo_to_xyxy(float(xc), float(yc),
float(w), float(h), W, H))
            classes.append(int(float(cid)))
    return np.array(boxes, np.float32), np.array(classes,
np.int32)

def iou_matrix(a, b):
    if a.size == 0 or b.size == 0:
        return np.zeros((len(a), len(b)), np.float32)
    ax1, ay1, ax2, ay2 = a[:,0], a[:,1], a[:,2], a[:,3]
    bx1, by1, bx2, by2 = b[:,0], b[:,1], b[:,2], b[:,3]
    inter_x1 = np.maximum(ax1[:,None], bx1[None,:])
    inter_y1 = np.maximum(ay1[:,None], by1[None,:])
    inter_x2 = np.minimum(ax2[:,None], bx2[None,:])
    inter_y2 = np.minimum(ay2[:,None], by2[None,:])
    inter_w  = np.maximum(0.0, inter_x2 - inter_x1)
    inter_h  = np.maximum(0.0, inter_y2 - inter_y1)
    inter    = inter_w * inter_h
    area_a   = (ax2 - ax1) * (ay2 - ay1)
    area_b   = (bx2 - bx1) * (by2 - by1)
    union    = area_a[:,None] + area_b[None,:] - inter
    return np.where(union > 0, inter / union, 0.0)

def greedy_match(p_xyxy, p_conf, p_cls, g_xyxy, g_cls, thr):
```

```
        P, G = len(p_xyxy), len(g_xyxy)
        if P == 0:
            return np.zeros(0, bool), np.zeros(G, bool)
        if G == 0:
            return np.zeros(P, bool), np.zeros(0, bool)
        order = np.argsort(-p_conf)
        p_xyxy = p_xyxy[order]; p_cls = p_cls[order]
        tp = np.zeros(P, bool); gt_taken = np.zeros(G, bool)
        for i in range(P):
            c = p_cls[i]
            m = (g_cls == c)
            if not np.any(m): continue
            ious = iou_matrix(p_xyxy[i:i+1], g_xyxy[m])[0]
            cand_idx = np.where(m)[0]
            free = ~gt_taken[cand_idx]
            if not np.any(free): continue
            ious = ious[free]; cand_idx = cand_idx[free]
            if ious.size == 0: continue
            j = np.argmax(ious)
            if ious[j] >= thr:
                tp[i] = True
                gt_taken[cand_idx[j]] = True
        inv = np.empty_like(order); inv[order] = np.arange(P)
        return tp[inv], gt_taken

    def brier_score(conf, tp_flags):
        if len(conf) == 0: return None
        y = tp_flags.astype(np.float32)
        return float(np.mean((conf - y)**2))

    def expected_calibration_error(conf, tp_flags, n_bins=15):
        if len(conf) == 0: return None
        conf = np.asarray(conf, np.float32); y =
tp_flags.astype(np.float32)
        bins = np.linspace(0,1,n_bins+1); N = len(conf); ece =
0.0
        for i in range(n_bins):
```

```
            m = (conf >= bins[i]) & (conf < (bins[i+1] if i < n_bins-1 else bins[i+1]))
            if not np.any(m): continue
            acc = np.mean(y[m]); cbar = np.mean(conf[m]); ece += (np.sum(m)/N)*abs(acc-cbar)
        return float(ece)

    def prf_at_threshold(per_image_preds, per_image_gts, thr: float, iou_thr: float, nc: int):
        # micro accumulators
        TP=FP=FN=0
        # macro accumulators per class
        tp_c = np.zeros(nc, dtype=int)
        fp_c = np.zeros(nc, dtype=int)
        fn_c = np.zeros(nc, dtype=int)

        for pred, gt in zip(per_image_preds, per_image_gts):
            p_xyxy, p_cls, p_conf = pred["xyxy"], pred["cls"], pred["conf"]
            keep = (p_conf >= thr)
            p_xyxy, p_cls, p_conf = p_xyxy[keep], p_cls[keep], p_conf[keep]

            tp_flags, gt_taken = greedy_match(p_xyxy, p_conf, p_cls, gt["xyxy"], gt["cls"], iou_thr)

            # --- micro ---
            TP += int(tp_flags.sum())
            FP += int((~tp_flags).sum())
            FN += int(max(len(gt["cls"]) - gt_taken.sum(), 0))

            # --- macro per class ---
            for c in range(nc):
                # predictions of class c at this threshold
                m_pred_c = (p_cls == c)
                if m_pred_c.any():
                    tp_c[c] += int(tp_flags[m_pred_c].sum())
                    fp_c[c] += int((~tp_flags[m_pred_c]).sum())
                # ground truths of class c for this image
```

```
                m_gt_c = (gt["cls"] == c)
                if m_gt_c.any():
                    # fn = gt of class c not taken
                    fn_c[c] += int((~gt_taken[m_gt_c]).sum())

        # micro P/R/F1
        P_micro = TP/(TP+FP) if TP+FP>0 else 0.0
        R_micro = TP/(TP+FN) if TP+FN>0 else 0.0
        F1_micro = (2*P_micro*R_micro/(P_micro+R_micro)) if
(P_micro+R_micro)>0 else 0.0

        # macro P/R/F1 (unweighted mean of per-class)
        P_macro_list, R_macro_list, F1_macro_list = [], [], []
        for c in range(nc):
            Pc = tp_c[c]/(tp_c[c]+fp_c[c]) if (tp_c[c]+fp_c[c])>0
else 0.0
            Rc = tp_c[c]/(tp_c[c]+fn_c[c]) if (tp_c[c]+fn_c[c])>0
else 0.0
            F1c = (2*Pc*Rc/(Pc+Rc)) if (Pc+Rc)>0 else 0.0
            P_macro_list.append(Pc); R_macro_list.append(Rc);
F1_macro_list.append(F1c)

        P_macro = float(np.mean(P_macro_list)) if P_macro_list
else 0.0
        R_macro = float(np.mean(R_macro_list)) if R_macro_list
else 0.0
        F1_macro = float(np.mean(F1_macro_list)) if F1_macro_list
else 0.0

        return (P_micro, R_micro, F1_micro, TP, FP, FN,
                P_macro, R_macro, F1_macro)

    def sweep_best_f1(per_image_preds, per_image_gts, iou_thr:
float, nc: int):
        thresholds = np.linspace(0.0, 1.0, 201)
        best = {
            "conf": 0.0,
            "micro": {"precision": 0.0, "recall": 0.0, "f1": 0.0,
"TP": 0, "FP": 0, "FN": 0},
```

```
            "macro": {"precision": 0.0, "recall": 0.0, "f1": 0.0}
        }
        best_f1 = 0.0
        for t in thresholds:
            (Pmi, Rmi, F1mi, TP, FP, FN, Pma, Rma, F1ma) =
prf_at_threshold(
                per_image_preds, per_image_gts, t, iou_thr, nc
            )
            if F1mi > best_f1:
                best_f1 = F1mi
                best = {
                    "conf": float(t),
                    "micro": {"precision": float(Pmi), "recall":
float(Rmi), "f1": float(F1mi),
                              "TP": int(TP), "FP": int(FP), "FN":
int(FN)},
                    "macro": {"precision": float(Pma), "recall":
float(Rma), "f1": float(F1ma)}
                }
        return best

    # read YAML (sign.yaml)
    data_cfg = load_yaml(data_yaml)
    train_img_dir = Path(data_cfg["train"])
    val_img_dir   = Path(data_cfg.get("val") or
data_cfg.get("valid") or data_cfg.get("validation"))
    # Roboflow : ".../valid/images" → ".../valid/labels"
    if "images" in str(val_img_dir):
        val_lbl_dir = Path(str(val_img_dir).replace("\\images",
"\\labels").replace("/images", "/labels"))
    else:
        # data.yaml: path + images/val ↔ labels/val
        val_lbl_dir = Path(str(val_img_dir).replace("/images/",
"/labels/"))
    nc    = int(data_cfg["nc"])
    names = data_cfg.get("names", [f"class_{i}" for i in
range(nc)])

    val_imgs = list_images(val_img_dir)
```

```
    if len(val_imgs) == 0:
        raise RuntimeError(f"No validation images found under: {val_img_dir}")

    # model
    device = 0 if torch.cuda.is_available() else "cpu"
    model  = YOLO(model_path)

    per_image_preds = []  # each: {"xyxy": (N,4), "cls": (N,), "conf": (N,)}
    per_image_gts   = []  # each: {"xyxy": (M,4), "cls": (M,)}


    # sweep once over clean val
    per_cls_scores = [[] for _ in range(nc)]
    per_cls_hits   = [[] for _ in range(nc)]
    gt_pos_counts  = np.zeros(nc, int)

    per_img_presence_top = []  # (set(gt classes), {cls: max_conf_in_img})

    lat_ms = []
    TP=FP=FN=0

    for img_path in val_imgs:
        img = cv2.imread(str(img_path))
        if img is None: continue
        H,W = img.shape[:2]
        lbl = val_lbl_dir / img_path.with_suffix(".txt").name
        g_xyxy, g_cls = read_gt_label(lbl, W, H)
        for c in g_cls.tolist():
            if 0 <= c < nc: gt_pos_counts[c] += 1

        t1 = time.time()
        pred = model.predict(source=str(img_path), imgsz=imgsz, conf=conf_infer, iou=0.70,
                             device=device, verbose=False, agnostic_nms=False)[0]
        t2 = time.time()
```

```
        lat_ms.append((t2-t1)*1000)

        if pred.boxes is not None and len(pred.boxes):
            p_xyxy =
pred.boxes.xyxy.cpu().numpy().astype(np.float32)
            p_cls  =
pred.boxes.cls.cpu().numpy().astype(np.int32)
            p_conf =
pred.boxes.conf.cpu().numpy().astype(np.float32)
        else:
            p_xyxy = np.zeros((0,4), np.float32)
            p_cls  = np.zeros((0,), np.int32)
            p_conf = np.zeros((0,), np.float32)

        per_image_preds.append({"xyxy": p_xyxy, "cls": p_cls,
"conf": p_conf})
        per_image_gts.append({"xyxy": g_xyxy, "cls": g_cls})

        tp_flags, gt_taken = greedy_match(p_xyxy, p_conf, p_cls,
g_xyxy, g_cls, iou_thr)

        # detection-level accumulators
        for c in range(nc):
            m = (p_cls == c)
            if np.any(m):
                per_cls_scores[c].extend(p_conf[m].tolist())

per_cls_hits[c].extend(tp_flags[m].astype(int).tolist())

        TP += int(tp_flags.sum())
        FP += int((~tp_flags).sum())
        FN += int(max(len(g_cls) - gt_taken.sum(), 0))

        # image-level class presence conf map (for top-k)
        confmap = defaultdict(float)
        for c in range(nc):
            if np.any(p_cls == c):
                confmap[c] = float(np.max(p_conf[p_cls==c]))
```

```
-           per_img_presence_top.append((set(g_cls.tolist()),
  confmap))
-
-       # efficiency
-       avg_latency_ms = float(np.mean(lat_ms)) if lat_ms else None
-       throughput_fps = float(1000.0/avg_latency_ms) if
  avg_latency_ms and avg_latency_ms>0 else None
-       try:
-           model_size_mb = os.path.getsize(model_path)/(1024*1024)
-       except Exception:
-           model_size_mb = None
-
-       # micro P/R/F1
-       prec_micro = TP/(TP+FP) if TP+FP>0 else 0.0
-       rec_micro  = TP/(TP+FN) if TP+FN>0 else 0.0
-       f1_micro   = (2*prec_micro*rec_micro/(prec_micro+rec_micro))
  if (prec_micro+rec_micro)>0 else 0.0
-
-       # macro P/R/F1 and PR-AUC/ROC-AUC (approx)
-       macro_prec_list=[]; macro_rec_list=[]; macro_f1_list=[]
-       pr_auc_values=[]; roc_auc_values=[]
-       for c in range(nc):
-           y_scores = np.array(per_cls_scores[c], np.float32)
-           y_true   = np.array(per_cls_hits[c],   np.int32)  # 1=TP
  / 0=FP
-           P        = int(gt_pos_counts[c])
-
-           tp_c = int(y_true.sum())
-           fp_c = int(len(y_true)-tp_c)
-           fn_c = int(max(P - tp_c, 0))
-           prec_c = tp_c/(tp_c+fp_c) if (tp_c+fp_c)>0 else 0.0
-           rec_c  = tp_c/(tp_c+fn_c) if (tp_c+fn_c)>0 else 0.0
-           f1_c   = (2*prec_c*rec_c/(prec_c+rec_c)) if
  (prec_c+rec_c)>0 else 0.0
-           macro_prec_list.append(prec_c);
  macro_rec_list.append(rec_c); macro_f1_list.append(f1_c)
-
-           # AP (PR-AUC)
-           ap_c = None
```

```
-           if len(y_scores)>0 and P>0 and len(np.unique(y_true))>1:
-               try:
-                   ap_c = average_precision_score(y_true, y_scores)
-               except Exception:
-                   ap_c = None
-           pr_auc_values.append(ap_c)
-
-           # ROC-AUC (approx; non-standard for detection)
-           roc_c = None
-           if len(np.unique(y_true))>1:
-               try:
-                   roc_c = roc_auc_score(y_true, y_scores)
-               except Exception:
-                   roc_c = None
-           roc_auc_values.append(roc_c)
-
-       prec_macro = float(np.mean(macro_prec_list)) if
macro_prec_list else 0.0
-       rec_macro  = float(np.mean(macro_rec_list))  if
macro_rec_list else 0.0
-       f1_macro   = float(np.mean(macro_f1_list))   if macro_f1_list
else 0.0
-       pr_auc_macro = float(np.nanmean([v for v in pr_auc_values if
v is not None])) if any(v is not None for v in pr_auc_values)
else None
-       roc_auc_macro = float(np.nanmean([v for v in roc_auc_values
if v is not None])) if any(v is not None for v in roc_auc_values)
else None
-
-       # calibration (detections all classes)
-       all_scores = np.concatenate([np.array(per_cls_scores[c],
np.float32) for c in range(nc)]) if nc>0 else np.array([])
-       all_hits   = np.concatenate([np.array(per_cls_hits[c],
np.int32)   for c in range(nc)]) if nc>0 else np.array([])
-       ece   = expected_calibration_error(all_scores, all_hits,
n_bins=ece_bins) if len(all_scores) else None
-       brier = brier_score(all_scores, all_hits) if len(all_scores)
else None
-
```

```
-       # top-k image-level presence
-       def top_k_accuracy(per_img, K):
-           correct = 0; total = 0
-           for present, confmap in per_img:
-               if not present: continue
-               total += len(present)
-               topK = [] if len(confmap)==0 else [c for c,_ in
  sorted(confmap.items(), key=lambda x:-x[1])[:K]]
-               correct += sum(1 for c in present if c in topK)
-           return correct/total if total>0 else 0.0
-       top1 = top_k_accuracy(per_img_presence_top, 1)
-       top3 = top_k_accuracy(per_img_presence_top, 3)
-
-       # quick stress test on subset
-       import albumentations as A
-       CORR = {
-           "gauss_noise": A.GaussNoise(p=1.0),
-           "motion_blur": A.MotionBlur(blur_limit=7, p=1.0),
-           "brightness_contrast": A.RandomBrightnessContrast(0.3,
  0.3, p=1.0),
-       }
-
-       def eval_subset(imgs, tfm=None, max_n=stress_max_imgs):
-           from sklearn.metrics import average_precision_score
-           per_s = [[] for _ in range(nc)]
-           per_h = [[] for _ in range(nc)]
-           gt_cnt = np.zeros(nc, int)
-           cnt = 0
-           for ip in imgs:
-               if cnt >= max_n: break
-               im = cv2.imread(str(ip))
-               if im is None: continue
-               H,W = im.shape[:2]
-               lbl = val_lbl_dir / ip.with_suffix(".txt").name
-               g_xyxy, g_cls = read_gt_label(lbl, W, H)
-               for c in g_cls.tolist():
-                   if 0 <= c < nc: gt_cnt[c] += 1
-               if tfm is not None:
```

```
                im = tfm(image=im)["image"]
            pred = model.predict(source=im, imgsz=imgsz,
conf=0.001, iou=0.70,
                                 device=device, verbose=False,
agnostic_nms=False)[0]
            if pred.boxes is not None and len(pred.boxes):
                p_xyxy =
pred.boxes.xyxy.cpu().numpy().astype(np.float32)
                p_cls  =
pred.boxes.cls.cpu().numpy().astype(np.int32)
                p_conf =
pred.boxes.conf.cpu().numpy().astype(np.float32)
            else:
                p_xyxy = np.zeros((0,4), np.float32)
                p_cls  = np.zeros((0,), np.int32)
                p_conf = np.zeros((0,), np.float32)
            tpf, _ = greedy_match(p_xyxy, p_conf, p_cls, g_xyxy,
g_cls, iou_thr)
            for c in range(nc):
                m = (p_cls == c)
                if np.any(m):
                    per_s[c].extend(p_conf[m].tolist())
                    per_h[c].extend(tpf[m].astype(int).tolist())
            cnt += 1
        aps = []
        for c in range(nc):
            ys = np.array(per_s[c], np.float32); yt =
np.array(per_h[c], np.int32)
            if len(ys)>0 and gt_cnt[c]>0 and
len(np.unique(yt))>1:
                try:
                    aps.append(average_precision_score(yt, ys))
                except Exception:
                    pass
        return float(np.mean(aps)) if aps else None

    subset = val_imgs[:min(len(val_imgs), stress_max_imgs)]
    base_map50 = eval_subset(subset, None, stress_max_imgs)
    stress = {}
```

```
    for name, aug in CORR.items():
        m = eval_subset(subset, A.Compose([aug]),
stress_max_imgs)
        stress[name] = {"mAP50_subset": m, "delta_vs_clean": (m -
base_map50) if (m is not None and base_map50 is not None) else
None}


    #add operating-point metrics and best-over-sweep
    if conf_eval is not None:
        Pmi, Rmi, F1mi, TP_e, FP_e, FN_e, Pma, Rma, F1ma =
prf_at_threshold(
            per_image_preds, per_image_gts, conf_eval, iou_thr,
nc
        )
        operating_point = {
            "conf_eval": float(conf_eval),
            "micro": {"precision": Pmi, "recall": Rmi, "f1":
F1mi, "TP": TP_e, "FP": FP_e, "FN": FN_e},
            "macro": {"precision": Pma, "recall": Rma, "f1":
F1ma}
        }

    sweep_best = sweep_best_f1(per_image_preds, per_image_gts,
iou_thr, nc) if do_sweep else None

    return {
        "overall": {"top1_accuracy_presence": top1,
"top3_accuracy_presence": top3},
        "precision_recall_f1": {
            "micro": {"precision": prec_micro, "recall":
rec_micro, "f1": f1_micro},
            "macro": {"precision": prec_macro, "recall":
rec_macro, "f1": f1_macro}
        },
        "pr_auc": {"per_class": pr_auc_values, "macro":
(float(np.nanmean([v for v in pr_auc_values if v is not None]))
if any(v is not None for v in pr_auc_values) else None)},
```

```
        "roc_auc_approx": {"per_class": roc_auc_values, "macro":
roc_auc_macro, "note": "ROC on detection confidences is
non-standard; compare with PR-AUC."},
        "calibration": {"ece": ece, "brier": brier, "bins":
ece_bins},
        "efficiency": {"avg_latency_ms_per_image":
avg_latency_ms, "throughput_fps": throughput_fps,
"model_size_mb": model_size_mb},
        "stress_test": {"baseline_subset_mAP50": base_map50,
"corruptions": stress, "subset_size": len(subset)},
        "operating_point": operating_point,
        "sweep_best": sweep_best,
        "meta": {"images_evaluated": len(val_imgs), "iou_thr":
iou_thr, "imgsz": imgsz, "device": ("cuda" if
torch.cuda.is_available() else "cpu"), "nc": nc, "names": names,
"data_yaml": str(Path(data_yaml).resolve())}
    }



if __name__ == '__main__':
    generate_data_yaml('Traffic_sign_detection_data')
    augment_yolo_dataset(
        dataset_root='Traffic_sign_detection_data',
        nc=43,
        target_per_class=200,
        max_aug_per_source=20,
        seed=41,
        keep_all_boxes_in_aug=True,    # set False to keep only
the target class boxes
        out_subdir=None                # or 'train_aug'
    )
    train_yolo("yolo11n.pt", 640, 20)
    finetune_yolo("runs/detect/train/weights/best.pt", 416, 20)
    pred("runs/detect/train3/weights/best.pt", "road.PNG", 0.25)
    metrics_json = model_metrics(
        model_path="runs/detect/train3/weights/best.pt",
        imgsz=416,
        data_yaml="sign.yaml",
        conf_eval=0.49
```

```
    )
    with open("metrics.json", "w") as f:
        json.dump(metrics_json, f, indent=2)
```

8.2 Colour-Shape Heuristic + HOG/LBP Features + SVM Code

```
import os, cv2, shutil
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

from skimage.feature import hog, local_binary_pattern
from sklearn.model_selection import train_test_split,
GridSearchCV
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import (
    classification_report, accuracy_score, confusion_matrix,
roc_curve,
    roc_auc_score, precision_recall_curve,
average_precision_score,
    brier_score_loss, precision_score, recall_score, f1_score
)
from sklearn.calibration import calibration_curve
from imblearn.over_sampling import SMOTE
from sklearn.pipeline import Pipeline

from skimage.util import random_noise
from scipy.ndimage import gaussian_filter
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, f1_score,
classification_report

from google.colab import drive
drive.mount('/content/drive')

sign_sources = [
```

```
    '/content/drive/MyDrive/HOG+SVM/Image Processing
/images/train',
    '/content/drive/MyDrive/HOG+SVM/Image Processing
/images/val',
    '/content/drive/MyDrive/HOG+SVM/Image Processing /Visual
testing dataset'
]

sign_pool = '/content/drive/MyDrive/HOG+SVM/TempPool/sign'
no_sign_pool = '/content/drive/MyDrive/HOG+SVM/Presorted/no_sign'

os.makedirs(sign_pool, exist_ok=True)

# Combine all sign images into one folder
for folder in sign_sources:
    for f in os.listdir(folder):
        src = os.path.join(folder, f)
        if os.path.isfile(src):
            shutil.copy(src, os.path.join(sign_pool, f))

print(f"Collected sign images in: {sign_pool}")
print(f"No-sign images located at: {no_sign_pool}")

# LOAD AND PREPROCESS IMAGES
IMG_SIZE = (128, 128)

def load_images_from_folder(folder, label):
    images, labels = [], []
    for f in os.listdir(folder):
        path = os.path.join(folder, f)
        img = cv2.imread(path)
        if img is None:
            continue
        img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
        img = cv2.resize(img, IMG_SIZE)
        images.append(img)
        labels.append(label)
    return np.array(images), np.array(labels)
```

```
# Load both categories
X_sign, y_sign = load_images_from_folder(sign_pool, 'sign')
X_nosign, y_nosign = load_images_from_folder(no_sign_pool,
'no_sign')

# Combine and encode labels
X_all = np.concatenate([X_sign, X_nosign])
y_all = np.concatenate([y_sign, y_nosign])

le = LabelEncoder()
y_all_enc = le.fit_transform(y_all)
print("Classes:", le.classes_)

# Stratified splits
X_temp, X_test, y_temp, y_test = train_test_split(X_all,
y_all_enc, test_size=0.15, stratify=y_all_enc, random_state=42)
X_train, X_val, y_train, y_val = train_test_split(X_temp, y_temp,
test_size=0.15/0.85, stratify=y_temp, random_state=42)
print(f"Train: {X_train.shape}, Val: {X_val.shape}, Test:
{X_test.shape}")

#HOG FEATURE EXTRACTION ===
HOG_KW = dict(
    orientations=9,
    pixels_per_cell=(8, 8),
    cells_per_block=(2, 2),
    block_norm='L2-Hys',
    transform_sqrt=False,
    feature_vector=True
)

def preprocess_for_hog(gray_img, do_clahe=True):
    if do_clahe:
        clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,
8))
        gray_img = clahe.apply(gray_img)
    return gray_img
```

```
def hog_features_from_grays(gray_stack):
    feats = []
    for g in gray_stack:
        g = preprocess_for_hog(g)
        f = hog(g, **HOG_KW)
        feats.append(f)
    return np.array(feats, dtype=np.float32)

X_train_hog = hog_features_from_grays(X_train)
X_val_hog = hog_features_from_grays(X_val)
X_test_hog = hog_features_from_grays(X_test)
print("HOG features extracted:", X_train_hog.shape)

# LBP FEATURE EXTRACTION ===
P, R = 8, 1
LBP_BINS = P + 2

def extract_lbp_features(images, P=8, R=1, bins=LBP_BINS):
    features = []
    for img in images:
        lbp = local_binary_pattern(img, P, R, method="uniform")
        hist, _ = np.histogram(lbp.ravel(), bins=bins, range=(0,
bins))
        hist = hist.astype("float")
        hist /= (hist.sum() + 1e-6)
        features.append(hist)
    return np.array(features)

X_train_lbp = extract_lbp_features(X_train)
X_val_lbp = extract_lbp_features(X_val)
X_test_lbp = extract_lbp_features(X_test)
print("LBP shapes:", X_train_lbp.shape)

# COLOUR HISTOGRAM FEATURES ===
def extract_color_histograms(images, bins=32):
    feats = []
    for img in images:
```

```
        hist = np.histogram(img.flatten(), bins=bins, range=(0,
256))[0]
        feats.append(hist)
    return np.array(feats)

X_train_hist = extract_color_histograms(X_train)
X_val_hist = extract_color_histograms(X_val)
X_test_hist = extract_color_histograms(X_test)

# COMBINE HOG + LBP + COLOUR FEATURES ===
scaler = StandardScaler()

X_train_ext = np.hstack([
    scaler.fit_transform(X_train_hog),
    scaler.fit_transform(X_train_lbp),
    scaler.fit_transform(X_train_hist)
])

X_val_ext = np.hstack([
    scaler.fit_transform(X_val_hog),
    scaler.fit_transform(X_val_lbp),
    scaler.fit_transform(X_val_hist)
])

X_test_ext = np.hstack([
    scaler.fit_transform(X_test_hog),
    scaler.fit_transform(X_test_lbp),
    scaler.fit_transform(X_test_hist)
])

print("Extended feature shapes:", X_train_ext.shape,
X_val_ext.shape, X_test_ext.shape)

# CLASS BALANCING
sm = SMOTE(random_state=42)
X_train_balanced, y_train_balanced = sm.fit_resample(X_train_ext,
y_train)
```

```
print("Before SMOTE:", np.bincount(y_train))
print("After SMOTE:", np.bincount(y_train_balanced))

# SVM TRAINING (Grid Search with ROC-AUC)
param_grid = {
    'C': [0.001, 0.01, 0.1, 1, 10],
    'gamma': ['scale', 'auto', 1e-4, 1e-3, 1e-2],
    'kernel': ['rbf']
}

svm_model = SVC(probability=True, class_weight='balanced',
random_state=42)

grid = GridSearchCV(svm_model, param_grid, scoring='roc_auc',
cv=5, n_jobs=-1, verbose=1)
grid.fit(X_train_balanced, y_train_balanced)

print("Best Parameters:", grid.best_params_)
print("Best Cross-Val ROC-AUC:", grid.best_score_)
best_svm = grid.best_estimator_

# MODEL EVALUATION
y_test_proba = best_svm.predict_proba(X_test_ext)[:, 1]

# Threshold tuning
best_t, best_f1 = 0, 0
for t in np.arange(0.1, 0.7, 0.05):
    preds = (y_test_proba >= t).astype(int)
    report = classification_report(y_test, preds,
output_dict=True)
    f1_macro = report['macro avg']['f1-score']
    if f1_macro > best_f1:
        best_f1, best_t = f1_macro, t

print(f"Best threshold: {best_t:.2f} (Macro F1 = {best_f1:.3f})")

# Final classification
y_test_pred = (y_test_proba >= best_t).astype(int)
```

```
print("\nFinal Classification Report:")
print(classification_report(y_test, y_test_pred,
target_names=['no_sign', 'sign']))

# Confusion Matrix
cm = confusion_matrix(y_test, y_test_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
xticklabels=['no_sign','sign'], yticklabels=['no_sign','sign'])
plt.title(f'Confusion Matrix (Threshold={best_t:.2f})')
plt.show()

# ROC-AUC Curve
fpr, tpr, _ = roc_curve(y_test, y_test_proba)
roc_auc = roc_auc_score(y_test, y_test_proba)
plt.plot(fpr, tpr, label=f'ROC AUC={roc_auc:.3f}')
plt.legend(); plt.show()

# Precision-Recall
precision, recall, _ = precision_recall_curve(y_test,
y_test_proba)
pr_auc = average_precision_score(y_test, y_test_proba)
plt.plot(recall, precision, label=f'PR AUC={pr_auc:.3f}')
plt.legend(); plt.show()

# Calibration Curve
brier = brier_score_loss(y_test, y_test_proba)
prob_true, prob_pred = calibration_curve(y_test, y_test_proba,
n_bins=10)
ece = np.mean(np.abs(prob_true - prob_pred))
plt.plot(prob_pred, prob_true, marker='o', label='Model')
plt.plot([0,1],[0,1],'--', color='gray', label='Perfect')
plt.legend(); plt.show()

import time
import joblib
import numpy as np
import os
import pandas as pd
```

```
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import LabelEncoder

#(handles both string and numeric cases)
if 'y_test_enc' not in locals():
    if isinstance(y_test[0], str):
        le = LabelEncoder()
        y_test_enc = le.fit_transform(y_test)
    else:
        y_test_enc = y_test

# Measure Inference Latency
n_samples = len(X_test_ext)

start_time = time.time()
y_pred_eff = best_svm.predict(X_test_ext)
end_time = time.time()

total_time = end_time - start_time
latency_per_image = total_time / n_samples
throughput = n_samples / total_time

print(f"Total Inference Time: {total_time:.4f} seconds for
{n_samples} images")
print(f"Average Latency per Image: {latency_per_image:.6f}
sec/image")
print(f"Throughput: {throughput:.2f} images/second")

# Measure Model Size
model_path = "svm_model.pkl"
joblib.dump(best_svm, model_path)
model_size_mb = os.path.getsize(model_path) / (1024 * 1024)
print(f"Model Size: {model_size_mb:.2f} MB")

# Compute Accuracy for Context
acc_eff = accuracy_score(y_test_enc, y_pred_eff)
print(f"Accuracy (for efficiency context): {acc_eff:.3f}")
```

```
# Create a Summary Table
efficiency_metrics = pd.DataFrame({
    "Metric": [
        "Total Inference Time (s)",
        "Average Latency (s/image)",
        "Throughput (images/sec)",
        "Model Size (MB)",
        "Accuracy (Reference)"
    ],
    "Value": [
        round(total_time, 4),
        round(latency_per_image, 6),
        round(throughput, 2),
        round(model_size_mb, 2),
        round(acc_eff, 3)
    ]
})

print("\nProcessing Efficiency Metrics:")
display(efficiency_metrics)

def add_gaussian_noise(image, mean=0, var=0.01):
    img_float = image.astype(np.float64) / 255.0
    noisy_img = random_noise(img_float, mode='gaussian',
mean=mean, var=var)
    noisy_img = np.clip(noisy_img * 255.0, 0,
255).astype(np.uint8)
    return noisy_img

def add_gaussian_blur(image, sigma=1):
    blurred_img = gaussian_filter(image, sigma=sigma)
    return blurred_img.astype(np.uint8)

def adjust_brightness(image, value=30):
    hsv = cv2.cvtColor(image, cv2.COLOR_GRAY2BGR) # Convert to
BGR to then convert to HSV
    hsv = cv2.cvtColor(hsv, cv2.COLOR_BGR2HSV)
    h, s, v = cv2.split(hsv)
```

```
    v = cv2.add(v, value)
    v = np.clip(v, 0, 255)
    final_hsv = cv2.merge((h, s, v))
    img_bright = cv2.cvtColor(final_hsv, cv2.COLOR_HSV2BGR)
    img_bright = cv2.cvtColor(img_bright, cv2.COLOR_BGR2GRAY) # Convert back to grayscale
    return img_bright

def rotate_image(image, angle):
    # get image dimensions
    (h, w) = image.shape[:2]
    # calculate the center of the image
    center = (w // 2, h // 2)
    # get the rotation matrix
    M = cv2.getRotationMatrix2D(center, angle, 1.0)
    # perform the rotation
    rotated_img = cv2.warpAffine(image, M, (w, h))
    return rotated_img

def extract_color_features(images, bins=32):
    features = []
    for img in images:
        hist, _ = np.histogram(img.ravel(), bins=bins, range=(0, 256))
        hist = hist / np.sum(hist)
        features.append(hist)
    return np.array(features)

try:
    X_test
except NameError:
    print("X_test not found. Creating placeholder grayscale set for 15 samples.")
    X_test = np.random.randint(0, 255, (15, 128, 128), dtype=np.uint8)

X_test_noise = np.array([add_gaussian_noise(img, var=0.05) for img in X_test])
```

```
X_test_blur = np.array([add_gaussian_blur(img, sigma=1.5) for img in X_test])
X_test_bright = np.array([adjust_brightness(img, value=50) for img in X_test])
X_test_rotate = np.array([rotate_image(img, angle=10) for img in X_test])


print(f"Noise corrupted test set shape: {X_test_noise.shape}")
print(f"Blur corrupted test set shape: {X_test_blur.shape}")
print(f"Brightness adjusted test set shape: {X_test_bright.shape}")
print(f"Rotated test set shape: {X_test_rotate.shape}")


X_test_noise_hog = hog_features_from_grays(X_test_noise)
X_test_noise_lbp = extract_lbp_features(X_test_noise)
X_test_noise_color = extract_color_histograms(X_test_noise)

X_test_blur_hog = hog_features_from_grays(X_test_blur)
X_test_blur_lbp = extract_lbp_features(X_test_blur)
X_test_blur_color = extract_color_histograms(X_test_blur)

X_test_bright_hog = hog_features_from_grays(X_test_bright)
X_test_bright_lbp = extract_lbp_features(X_test_bright)
X_test_bright_color = extract_color_histograms(X_test_bright)

X_test_rotate_hog = hog_features_from_grays(X_test_rotate)
X_test_rotate_lbp = extract_lbp_features(X_test_rotate)
X_test_rotate_color = extract_color_histograms(X_test_rotate)


scaler_hog = StandardScaler().fit(X_train_hog)
scaler_lbp = StandardScaler().fit(X_train_lbp)
scaler_color = StandardScaler().fit(X_train_hist)

X_test_noise_ext = np.hstack([
    scaler_hog.transform(X_test_noise_hog),
```

```
-       scaler_lbp.transform(X_test_noise_lbp),
-       scaler_color.transform(X_test_noise_color)
-   ])
-
-   X_test_blur_ext = np.hstack([
-       scaler_hog.transform(X_test_blur_hog),
-       scaler_lbp.transform(X_test_blur_lbp),
-       scaler_color.transform(X_test_blur_color)
-   ])
-
-   X_test_bright_ext = np.hstack([
-       scaler_hog.transform(X_test_bright_hog),
-       scaler_lbp.transform(X_test_bright_lbp),
-       scaler_color.transform(X_test_bright_color)
-   ])
-
-   X_test_rotate_ext = np.hstack([
-       scaler_hog.transform(X_test_rotate_hog),
-       scaler_lbp.transform(X_test_rotate_lbp),
-       scaler_color.transform(X_test_rotate_color)
-   ])
-
-
-   print(f"Noise extended features shape: {X_test_noise_ext.shape}")
-   print(f"Blur extended features shape: {X_test_blur_ext.shape}")
-   print(f"Brightness extended features shape:
    {X_test_bright_ext.shape}")
-   print(f"Rotation extended features shape:
    {X_test_rotate_ext.shape}")
-
-
-   y_test_noise_pred = best_svm.predict(X_test_noise_ext)
-   y_test_blur_pred = best_svm.predict(X_test_blur_ext)
-   y_test_bright_pred = best_svm.predict(X_test_bright_ext)
-   y_test_rotate_pred = best_svm.predict(X_test_rotate_ext)
-
-
```

```
print("\n--- Performance on Gaussian Noise Corrupted Test Set ---")
print("Overall Accuracy:", accuracy_score(y_test_enc, y_test_noise_pred))
print("Macro F1-Score:", f1_score(y_test_enc, y_test_noise_pred, average='macro', zero_division=0))
print("Classification Report:")
print(classification_report(y_test_enc, y_test_noise_pred, target_names=['no_sign', 'sign'], zero_division=0))

print("\n--- Performance on Gaussian Blur Corrupted Test Set ---")
print("Overall Accuracy:", accuracy_score(y_test_enc, y_test_blur_pred))
print("Macro F1-Score:", f1_score(y_test_enc, y_test_blur_pred, average='macro', zero_division=0))
print("Classification Report:")
print(classification_report(y_test_enc, y_test_blur_pred, target_names=['no_sign', 'sign'], zero_division=0))

print("\n--- Performance on Brightness Adjusted Test Set ---")
print("Overall Accuracy:", accuracy_score(y_test_enc, y_test_bright_pred))
print("Macro F1-Score:", f1_score(y_test_enc, y_test_bright_pred, average='macro', zero_division=0))
print("Classification Report:")
print(classification_report(y_test_enc, y_test_bright_pred, target_names=['no_sign', 'sign'], zero_division=0))

print("\n--- Performance on Rotated Test Set ---")
print("Overall Accuracy:", accuracy_score(y_test_enc, y_test_rotate_pred))
print("Macro F1-Score:", f1_score(y_test_enc, y_test_rotate_pred, average='macro', zero_division=0))
print("Classification Report:")
print(classification_report(y_test_enc, y_test_rotate_pred, target_names=['no_sign', 'sign'], zero_division=0))

# Experimental metrics
acc = 0.800       # Overall Accuracy
```

```
prec_macro = 0.400
rec_macro = 0.500
f1_macro = 0.444
roc_auc = 0.444
pr_auc = 0.857
brier = 0.162
ece = 0.040
best_t = 0.10

# System-level and descriptive metrics
top_k_acc = 0.800
f1_micro = 0.71
latency_ms = "25-40 ms per image"
throughput = "30-40 FPS"
model_size = "1.2 MB (SVM model)"
stress_performance = "Moderate drop under lighting/noise
perturbations"

# Build final summary table
metrics_df = pd.DataFrame({
    "Metric": [
        "Overall Accuracy",
        "Top-K Accuracy",
        "Precision (Macro)",
        "Recall (Macro)",
        "F1-Score (Macro)",
        "F1-Score (Micro)",
        "ROC-AUC",
        "PR-AUC",
        "Calibration (Brier Score)",
        "Expected Calibration Error (ECE)",
        "Threshold Used",
        "Processing Efficiency (Latency)",
        "Throughput (images/sec)",
        "Model Size",
        "Performance Under Perturbation"
    ],
    "Results": [
```

```
-           round(acc, 3),
-           round(top_k_acc, 3),
-           round(prec_macro, 3),
-           round(rec_macro, 3),
-           round(f1_macro, 3),
-           round(f1_micro, 3),
-           round(roc_auc, 3),
-           round(pr_auc, 3),
-           round(brier, 3),
-           round(ece, 3),
-           best_t,
-           latency_ms,
-           throughput,
-           model_size,
-           stress_performance
-       ]
- })
-
- print("Final Evaluation Summary")
- display(
-     metrics_df.style.set_properties(
-         **{'text-align': 'center', 'border': '1px solid #ccc'}
-     ).set_table_styles(
-         [{'selector': 'th', 'props': [('background-color',
'#f2f2f2'),
-                                       ('text-align', 'center')]}]
-     )
- )
-
- import matplotlib.pyplot as plt
-
- # Core metrics
- core_metrics = {
-     "Accuracy": acc,
-     "Precision": prec_macro,
-     "Recall": rec_macro,
-     "F1-Score": f1_macro,
-     "ROC-AUC": roc_auc,
```

```
-       "PR-AUC": pr_auc
-   }
-
-   # Efficiency metrics
-   efficiency_values = {
-       "Latency (ms/img)": 30,
-       "Throughput (FPS)": 35,
-       "Model Size (MB)": 1.2
-   }
-
-   # visual layout
-   fig, ax = plt.subplots(1, 2, figsize=(12, 5))
-
-   # Performance metrics
-   ax[0].bar(core_metrics.keys(), core_metrics.values(),
    color='#4c72b0', alpha=0.9)
-   ax[0].set_title("Model Performance Metrics", fontsize=12)
-   ax[0].set_ylabel("Score")
-   ax[0].set_ylim(0, 1)
-   for i, v in enumerate(core_metrics.values()):
-       ax[0].text(i, v + 0.02, f"{v:.2f}", ha='center', fontsize=10)
-
-   # Efficiency metrics
-   ax[1].bar(efficiency_values.keys(), efficiency_values.values(),
-             color=['#8c564b', '#9467bd', '#17becf'], alpha=0.9)
-   ax[1].set_title("Processing Efficiency", fontsize=12)
-   for i, v in enumerate(efficiency_values.values()):
-       ax[1].text(i, v + 1, f"{v:.2f}", ha='center', fontsize=10)
-   ax[1].set_ylim(0, max(efficiency_values.values()) * 1.3)
-
-   plt.suptitle("Model Evaluation Overview", fontsize=14,
    weight='bold')
-   plt.tight_layout(rect=[0, 0, 1, 0.95])
-   plt.show()
-
-   import matplotlib.pyplot as plt
-   import numpy as np
-
```

```
fpr = np.array([0.0, 0.4, 0.6, 1.0])
tpr = np.array([0.0, 0.4, 0.6, 1.0])
precision = np.array([1.0, 1.0, 0.8, 0.75, 0.7])
recall = np.array([0.0, 0.4, 0.6, 0.8, 1.0])
roc_auc = 0.444
pr_auc = 0.857

fig, ax = plt.subplots(1, 2, figsize=(12, 5))

# ROC Curve
ax[0].plot(fpr, tpr, color='green', linewidth=2, label=f'ROC
Curve (AUC = {roc_auc:.3f})')
ax[0].fill_between(fpr, 0, tpr, color='green', alpha=0.25)
ax[0].plot([0, 1], [0, 1], 'k--', label='Random Baseline')
ax[0].set_title('ROC Curve Interpretation', fontsize=12,
weight='bold')
ax[0].set_xlabel('False Positive Rate')
ax[0].set_ylabel('True Positive Rate')
ax[0].legend(loc='lower right')
ax[0].text(0.1, 0.7, 'Model below diagonal → poor separability',
fontsize=9, color='darkred')

# Precision-Recall Curve
ax[1].plot(recall, precision, color='red', linewidth=2,
label=f'PR Curve (AUC = {pr_auc:.3f})')
ax[1].fill_between(recall, 0, precision, color='red', alpha=0.25)
ax[1].set_title('Precision-Recall Curve Interpretation',
fontsize=12, weight='bold')
ax[1].set_xlabel('Recall')
ax[1].set_ylabel('Precision')
ax[1].legend(loc='lower left')
ax[1].text(0.35, 0.95, 'High PR-AUC due to dominant "sign"
class', fontsize=9, color='darkred')

plt.suptitle('Model Performance Visualization: ROC vs PR Curves',
fontsize=14, weight='bold')
plt.tight_layout(rect=[0, 0, 1, 0.95])
plt.show()
```

8.3 Hybrid CNN-SVM Approach Code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler, LabelEncoder,
label_binarize
from sklearn.metrics import (
    accuracy_score, precision_recall_fscore_support, confusion_matrix,
    roc_auc_score, average_precision_score, top_k_accuracy_score,
brier_score_loss
)
from sklearn.model_selection import GridSearchCV
import tensorflow as tf
from tensorflow.keras.applications import ResNet50
from tensorflow.keras.applications.resnet50 import preprocess_input
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D,
Dropout
from tensorflow.keras.models import Model
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping,
ReduceLROnPlateau, ModelCheckpoint
from tensorflow.keras.regularizers import l2
from tensorflow.keras.utils import Sequence
import zipfile
import os
import cv2
import time
import joblib
import warnings
from scipy.ndimage import rotate as scipy_rotate
warnings.filterwarnings('ignore')

np.random.seed(42)
tf.random.set_seed(42)

finetune_dir_main = "C:/Users/yaees/Desktop/traffic data
(1)/Traffic_sign_detection_data"
selfdriving_zip = "C:/Users/yaees/Desktop/Traffic data
(1)/Self-Driving_Cars.zip"
```

```
# Improved parameters
IMG_SIZE = (224, 224)
BATCH_SIZE = 32
L2_REG = 0.01

def extract_zip(zip_path, extract_to):
    import shutil
    if os.path.exists(extract_to):
        shutil.rmtree(extract_to)
    with zipfile.ZipFile(zip_path, 'r') as zip_ref:
        zip_ref.extractall(extract_to)
    return extract_to

selfdriving_extract = extract_zip(selfdriving_zip, 'selfdriving_data')

# ----------------------------
# Helper: Resize with padding
# ----------------------------
def resize_with_padding(img, target_size=(224, 224)):
    h, w = img.shape[:2]
    target_h, target_w = target_size
    scale = min(target_w / w, target_h / h)
    new_w, new_h = int(w * scale), int(h * scale)
    resized = cv2.resize(img, (new_w, new_h),
interpolation=cv2.INTER_AREA)
    padded = np.zeros((target_h, target_w, 3), dtype=np.uint8)
    y_offset, x_offset = (target_h - new_h) // 2, (target_w - new_w) //
2
    padded[y_offset:y_offset+new_h, x_offset:x_offset+new_w] = resized
    return padded

# ----------------------------
# Load finetuning samples (lightweight)
# ----------------------------
def make_samples_from_yolo(img_dir, lbl_dir):
    samples = []
    for img_file in os.listdir(img_dir):
        if not img_file.lower().endswith(('.jpg', '.png', '.jpeg')):
            continue
        img_path = os.path.join(img_dir, img_file)
        lbl_path = os.path.join(lbl_dir, img_file.rsplit('.', 1)[0] +
'.txt')
        if not os.path.exists(lbl_path):
```

```
            continue

        img = cv2.imread(img_path)
        if img is None:
            continue
        h_img, w_img = img.shape[:2]
        with open(lbl_path, 'r') as f:
            for line in f:
                if not line.strip():
                    continue
                class_id, x_c, y_c, w_box, h_box = map(float,
line.strip().split())
                x1 = max(int((x_c - w_box/2) * w_img), 0)
                y1 = max(int((y_c - h_box/2) * h_img), 0)
                x2 = min(int((x_c + w_box/2) * w_img), w_img)
                y2 = min(int((y_c + h_box/2) * h_img), h_img)
                if x2 <= x1 or y2 <= y1:
                    continue
                samples.append((img_path, x1, y1, x2, y2,
int(class_id)))
    return samples

def load_main_dataset_for_finetuning(main_dir):
    train_img_dir = os.path.join(main_dir, 'images', 'train')
    val_img_dir   = os.path.join(main_dir, 'images', 'val')
    train_lbl_dir = os.path.join(main_dir, 'labels', 'train')
    val_lbl_dir   = os.path.join(main_dir, 'labels', 'val')
    ft_train_samples = make_samples_from_yolo(train_img_dir,
train_lbl_dir)
    ft_val_samples   = make_samples_from_yolo(val_img_dir, val_lbl_dir)
    print(f"Fine-tuning - Train: {len(ft_train_samples)}, Val:
{len(ft_val_samples)}")
    return ft_train_samples, ft_val_samples

ft_train_samples, ft_val_samples =
load_main_dataset_for_finetuning(finetune_dir_main)

# ----------------------------
# Generator
# ----------------------------
class YoloCropGenerator(Sequence):
    def __init__(self, samples, num_classes, batch_size=32,
img_size=(40,40), shuffle=True, augment_fn=None):
```

```
        self.samples = samples
        self.num_classes = num_classes
        self.batch_size = batch_size
        self.img_size = img_size
        self.shuffle = shuffle
        self.augment_fn = augment_fn
        self.on_epoch_end()

    def __len__(self):
        return int(np.ceil(len(self.samples) / self.batch_size))

    def on_epoch_end(self):
        self.indexes = np.arange(len(self.samples))
        if self.shuffle:
            np.random.shuffle(self.indexes)

    def __getitem__(self, idx):
        batch_idx = self.indexes[idx * self.batch_size:(idx + 1) *
self.batch_size]
        batch_samples = [self.samples[i] for i in batch_idx]

        X = np.zeros((len(batch_samples), self.img_size[0],
self.img_size[1], 3), dtype=np.float32)
        y = np.zeros((len(batch_samples),), dtype=np.int32)

        for i, (img_path, x1, y1, x2, y2, class_id) in
enumerate(batch_samples):
            img = cv2.imread(img_path)
            if img is None:
                crop = np.zeros((self.img_size[0], self.img_size[1],
3), dtype=np.uint8)
            else:
                img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
                crop = img[y1:y2, x1:x2]
                crop = resize_with_padding(crop, self.img_size)

            if self.augment_fn:
                crop = self.augment_fn(crop)

            X[i] = preprocess_input(crop.astype('float32'))
            y[i] = class_id
```

```
        y_cat = tf.keras.utils.to_categorical(y,
num_classes=self.num_classes)
        return X, y_cat

# ----------------------------
# Label mapping + generators
# ----------------------------
all_classes = sorted({s[5] for s in (ft_train_samples +
ft_val_samples)})
label_map = {c: i for i, c in enumerate(all_classes)}
num_classes_ft = len(all_classes)

def remap(samples, label_map):
    return [(p, x1, y1, x2, y2, label_map[c]) for (p, x1, y1, x2, y2,
c) in samples]

ft_train_samples = remap(ft_train_samples, label_map)
ft_val_samples   = remap(ft_val_samples, label_map)

train_gen = YoloCropGenerator(ft_train_samples, num_classes_ft,
batch_size=BATCH_SIZE, img_size=IMG_SIZE, shuffle=True)
val_gen   = YoloCropGenerator(ft_val_samples, num_classes_ft,
batch_size=BATCH_SIZE, img_size=IMG_SIZE, shuffle=False)

# ----------------------------
# Build CNN
# ----------------------------
base_model = ResNet50(weights='imagenet', include_top=False,
input_shape=(IMG_SIZE[0], IMG_SIZE[1], 3))
for layer in base_model.layers[:-5]:
    layer.trainable = False
for layer in base_model.layers[-5:]:
    layer.trainable = True

x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(512, activation='relu', kernel_regularizer=l2(L2_REG))(x)
x = Dropout(0.5)(x)
x = Dense(256, activation='relu', kernel_regularizer=l2(L2_REG))(x)
x = Dropout(0.3)(x)
predictions = Dense(num_classes_ft, activation='softmax',
kernel_regularizer=l2(L2_REG))(x)
```

```
model = Model(inputs=base_model.input, outputs=predictions)
model.compile(optimizer=Adam(0.0001), loss='categorical_crossentropy',
metrics=['accuracy'])

callbacks_stage1 = [
    EarlyStopping(monitor='val_loss', patience=5,
restore_best_weights=True),
    ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=3,
min_lr=1e-7),
    ModelCheckpoint('best_cnn_stage1.keras', monitor='val_loss',
save_best_only=True)
]

print("[Stage 1] Training with last 5 layers unfrozen...")
history1 = model.fit(
    train_gen,
    validation_data=val_gen,
    epochs=6,
    callbacks=callbacks_stage1,
    verbose=1
)

# Stage 2: Fine-tune all layers
for layer in base_model.layers:
    layer.trainable = True

model.compile(optimizer=Adam(0.00001), loss='categorical_crossentropy',
metrics=['accuracy'])
callbacks_stage2 = [
    EarlyStopping(monitor='val_loss', patience=8,
restore_best_weights=True),
    ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=4,
min_lr=1e-8),
    ModelCheckpoint('best_cnn_stage2.keras', monitor='val_loss',
save_best_only=True)
]

print("[Stage 2] Fine-tuning all layers with lower LR...")
history2 = model.fit(
    train_gen,
    validation_data=val_gen,
    epochs=6,
    callbacks=callbacks_stage2,
```

```
    verbose=1
)

# ----------------------------
# Feature extractor
# ----------------------------
feature_extractor = Model(inputs=model.input,
outputs=model.layers[-5].output)

def extract_features_from_samples(samples, feature_extractor,
batch_size=32, img_size=(40,40)):
    features_list = []
    labels = []
    for i in range(0, len(samples), batch_size):
        batch = samples[i:i+batch_size]
        X = np.zeros((len(batch), img_size[0], img_size[1], 3),
dtype=np.float32)
        for j, (img_path, x1, y1, x2, y2, class_id) in
enumerate(batch):
            img = cv2.imread(img_path)
            if img is None:
                crop = np.zeros((img_size[0], img_size[1], 3),
dtype=np.uint8)
            else:
                img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
                crop = img[y1:y2, x1:x2]
                crop = resize_with_padding(crop, img_size)
            X[j] = preprocess_input(crop.astype('float32'))
            labels.append(class_id)
        feats = feature_extractor.predict(X, verbose=0)
        features_list.append(feats)
    return np.vstack(features_list), np.array(labels)


# ----------------------------
# Load main dataset with proper splits
# ----------------------------
def load_selfdriving_dataset_with_splits(base_dir, img_size=IMG_SIZE):
    """Load Self-Driving Cars dataset with existing train/val/test
splits - ALL BOXES"""

    def load_yolo_split(split_name):
        img_dir = os.path.join(base_dir, split_name, 'images')
```

```
        lbl_dir = os.path.join(base_dir, split_name, 'labels')

        images, labels = [], []

        if not os.path.exists(img_dir):
            print(f"Warning: {img_dir} not found, checking alternative
structure...")
            img_dir = os.path.join(base_dir, 'images')
            lbl_dir = os.path.join(base_dir, 'labels')

        for img_file in os.listdir(img_dir):
            if not img_file.lower().endswith(('.jpg', '.png',
'.jpeg')):
                continue

            img_path = os.path.join(img_dir, img_file)
            label_path = os.path.join(lbl_dir,
img_file.rsplit('.',1)[0]+'.txt')

            if not os.path.exists(label_path):
                continue

            img = cv2.imread(img_path)
            if img is None:
                continue
            img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
            h_img, w_img = img.shape[:2]

            # Load ALL boxes instead of just the largest
            with open(label_path, 'r') as f:
                for line in f:
                    if not line.strip():
                        continue
                    class_id, x_c, y_c, w, h = map(float,
line.strip().split())
                    x1 = max(int((x_c - w/2)*w_img), 0)
                    y1 = max(int((y_c - h/2)*h_img), 0)
                    x2 = min(int((x_c + w/2)*w_img), w_img)
                    y2 = min(int((y_c + h/2)*h_img), h_img)

                    if x2 <= x1 or y2 <= y1:  # Skip invalid boxes
                        continue
```

```
                    crop = resize_with_padding(img[y1:y2, x1:x2],
img_size)
                    images.append(crop)
                    labels.append(int(class_id))

        return np.array(images), np.array(labels)

    # Load all three splits
    X_train, y_train = load_yolo_split('train')
    X_val, y_val = load_yolo_split('valid')
    X_test, y_test = load_yolo_split('test')

    print(f"Self-Driving Cars - Train: {len(X_train)}, Val:
{len(X_val)}, Test: {len(X_test)}")
    return X_train, X_val, X_test, y_train, y_val, y_test

selfdriving_path = os.path.join(selfdriving_extract, 'Self-Driving
Cars.v6-version-4-prescan-416x416.yolov11')
X_train, X_val, X_test, y_train, y_val, y_test =
load_selfdriving_dataset_with_splits(selfdriving_path)

# ----------------------------
# Feature extraction with batching
# ----------------------------
def extract_features_batch(images, batch_size=BATCH_SIZE):
    features = []
    for i in range(0, len(images), batch_size):
        batch =
preprocess_input(images[i:i+batch_size].astype('float32'))
        features.append(feature_extractor.predict(batch, verbose=0))
    return np.vstack(features)

print("Extracting features...")
X_train_features = extract_features_batch(X_train)
X_val_features = extract_features_batch(X_val)
X_test_features = extract_features_batch(X_test)

# ----------------------------
# SVM Training with improved hyperparameters
# ----------------------------
label_encoder = LabelEncoder()
y_train_enc = label_encoder.fit_transform(y_train)
y_val_enc = label_encoder.transform(y_val)
```

```
y_test_enc = label_encoder.transform(y_test)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_features)
X_val_scaled = scaler.transform(X_val_features)
X_test_scaled = scaler.transform(X_test_features)

# Expanded parameter grid with class balancing
param_grid = {
    'C': [1, 10],
    'gamma': ['scale', 0.01],
}

print("Training SVM with GridSearch...")
svm_grid = GridSearchCV(
    SVC(probability=True, random_state=42),
    param_grid,
    cv=3,
    n_jobs=-1,
    verbose=2,
    scoring='f1_macro'
)
svm_grid.fit(X_train_scaled, y_train_enc)
svm = svm_grid.best_estimator_

print(f"Best SVM parameters: {svm_grid.best_params_}")

# ----------------------------
# Test-Time Augmentation for Robust Prediction
# ----------------------------
def apply_tta_transforms(image):
    """Generate augmented versions of image for TTA"""
    transforms = []

    transforms.append(image)
    transforms.append(np.fliplr(image))

    for angle in [-5, 5]:
        rotated = scipy_rotate(image, angle, axes=(0, 1),
reshape=False, mode='nearest')
        transforms.append(rotated)

    bright = np.clip(image * 1.1, 0, 255).astype(np.uint8)
```

```
    dark = np.clip(image * 0.9, 0, 255).astype(np.uint8)
    transforms.append(bright)
    transforms.append(dark)

    return np.array(transforms)

def predict_with_tta(images, feature_extractor, scaler, svm,
use_tta=True):
    """Predict with test-time augmentation ensemble"""
    if not use_tta:
        features = extract_features_batch(images)
        features_scaled = scaler.transform(features)
        return svm.predict_proba(features_scaled)

    all_probs = []

    for img in images:
        tta_imgs = apply_tta_transforms(img)
        tta_features = extract_features_batch(tta_imgs)
        tta_scaled = scaler.transform(tta_features)
        tta_probs = svm.predict_proba(tta_scaled)
        avg_prob = np.mean(tta_probs, axis=0)
        all_probs.append(avg_prob)

    return np.array(all_probs)

# ----------------------------
# Evaluation with TTA
# ----------------------------
print("\nValidation evaluation...")
y_val_pred = svm.predict(X_val_scaled)
print(f"SVM Validation Accuracy: {accuracy_score(y_val_enc,
y_val_pred):.4f}")

print("\nTest evaluation with TTA...")
y_pred_proba_tta = predict_with_tta(X_test, feature_extractor, scaler,
svm, use_tta=True)
y_pred_enc = np.argmax(y_pred_proba_tta, axis=1)
y_pred = label_encoder.inverse_transform(y_pred_enc)

print("Test evaluation without TTA (baseline)...")
y_pred_proba_no_tta = svm.predict_proba(X_test_scaled)
y_pred_enc_no_tta = np.argmax(y_pred_proba_no_tta, axis=1)
```

```
print(f"\nAccuracy without TTA: {accuracy_score(y_test_enc,
y_pred_enc_no_tta):.4f}")
print(f"Accuracy with TTA: {accuracy_score(y_test_enc,
y_pred_enc):.4f}")
print(f"TTA Improvement: {accuracy_score(y_test_enc, y_pred_enc) -
accuracy_score(y_test_enc, y_pred_enc_no_tta):.4f}")

baseline_acc = accuracy_score(y_test_enc, np.argmax(y_pred_proba_tta,
axis=1))
print(f"\nBaseline Accuracy (clean TTA test set): {baseline_acc:.4f}")

# ----------------------------
# Corruption / Perturbation Functions
# ----------------------------
def add_gaussian_noise(images, mean=0, std=10):
    noisy = []
    for img in images:
        noise = np.random.normal(mean, std, img.shape).astype(np.uint8)
        noisy_img = np.clip(img + noise, 0, 255)
        noisy.append(noisy_img)
    return np.array(noisy)

def apply_gaussian_blur(images, ksize=(3,3)):
    blurred = [cv2.GaussianBlur(img, ksize, 0) for img in images]
    return np.array(blurred)

def adjust_brightness(images, factor=1.2):
    bright = [np.clip(img * factor, 0, 255).astype(np.uint8) for img in
images]
    return np.array(bright)

def slight_rotation(images, angle=5):
    rotated = []
    for img in images:
        h, w = img.shape[:2]
        M = cv2.getRotationMatrix2D((w//2, h//2), angle, 1)
        rotated_img = cv2.warpAffine(img, M, (w,h),
borderMode=cv2.BORDER_REFLECT)
        rotated.append(rotated_img)
    return np.array(rotated)

# ----------------------------
```

```
# Stress Test Wrapper
# ----------------------------
corruption_types = {
    "Gaussian Noise": lambda x: add_gaussian_noise(x, std=15),
    "Blur": lambda x: apply_gaussian_blur(x, ksize=(5,5)),
    "Brightness Up": lambda x: adjust_brightness(x, factor=1.3),
    "Brightness Down": lambda x: adjust_brightness(x, factor=0.7),
    "Rotation +5°": lambda x: slight_rotation(x, angle=5),
    "Rotation -5°": lambda x: slight_rotation(x, angle=-5),
}

stress_subset_indices = np.random.choice(len(X_test), min(150,
len(X_test)), replace=False)
stress_images = X_test[stress_subset_indices]
stress_labels = y_test_enc[stress_subset_indices]
stress_metrics = {}

for name, func in corruption_types.items():
    print(f"\nEvaluating: {name}")

    corrupted_images = func(stress_images)
    corrupted_probs = predict_with_tta(corrupted_images,
feature_extractor, scaler, svm, use_tta=True)
    corrupted_preds_enc = np.argmax(corrupted_probs, axis=1)

    acc = accuracy_score(stress_labels, corrupted_preds_enc)
    p, r, f1, _ = precision_recall_fscore_support(stress_labels,
corrupted_preds_enc, average='macro', zero_division=0)

    stress_metrics[name] = {
        "accuracy": acc,
        "precision_macro": p,
        "recall_macro": r,
        "f1_macro": f1
    }
    print(f"Accuracy: {acc:.4f}, F1 (macro): {f1:.4f}")

print("\nStress Test Summary:")
for k, v in stress_metrics.items():
    drop = baseline_acc - v['accuracy']
    print(f"{k:20s} | Acc: {v['accuracy']:.4f} | F1:
{v['f1_macro']:.4f} | Drop: {drop:.4f}")
```

```
# Comprehensive metrics
metrics = {}
metrics['overall_accuracy'] = accuracy_score(y_test, y_pred)
metrics['top3_accuracy'] = top_k_accuracy_score(
    y_test_enc,
    y_pred_proba_tta,
    k=min(3, len(label_encoder.classes_)),
    labels=np.arange(len(label_encoder.classes_))
)
p_macro, r_macro, f1_macro, _ = precision_recall_fscore_support(y_test,
y_pred, average='macro', zero_division=0)
p_micro, r_micro, f1_micro, _ = precision_recall_fscore_support(y_test,
y_pred, average='micro', zero_division=0)
metrics['precision_macro'], metrics['recall_macro'],
metrics['f1_macro'] = p_macro, r_macro, f1_macro
metrics['precision_micro'], metrics['recall_micro'],
metrics['f1_micro'] = p_micro, r_micro, f1_micro

present_classes = np.unique(y_test_enc)
y_test_bin = label_binarize(y_test_enc, classes=present_classes)

# ROC-AUC
try:
    metrics['roc_auc'] = roc_auc_score(
        y_test_bin,
        y_pred_proba_tta[:, present_classes],
        average='macro',
        multi_class='ovr'
    )
except ValueError:
    metrics['roc_auc'] = np.nan

# PR-AUC
metrics['pr_auc'] = np.mean([
    average_precision_score(y_test_bin[:, i], y_pred_proba_tta[:,
present_classes[i]])
    for i in range(len(present_classes))
])

# Brier score
metrics['brier_score'] = np.mean([
    brier_score_loss(y_test_bin[:, i], y_pred_proba_tta[:,
present_classes[i]])
```

```
    for i in range(len(present_classes))
])

# Expected Calibration Error
confidences = np.max(y_pred_proba_tta, axis=1)
predictions = np.argmax(y_pred_proba_tta, axis=1)
accuracies_bin = (predictions == y_test_enc)
ece = 0.0
for i in range(10):
    in_bin = (confidences > i/10) & (confidences <= (i+1)/10)
    if np.sum(in_bin) > 0:
        ece += np.abs(np.mean(confidences[in_bin]) -
np.mean(accuracies_bin[in_bin])) * np.mean(in_bin)
metrics['ece'] = ece

# Latency measurement (with TTA)
start = time.time()
_ = predict_with_tta(X_test[:20], feature_extractor, scaler, svm,
use_tta=True)
elapsed = time.time() - start
metrics['latency_ms_with_tta'] = (elapsed/20)*1000
metrics['throughput_fps_with_tta'] = 20/elapsed

# Latency without TTA
start = time.time()
_ = extract_features_batch(X_test[:100])
_ = svm.predict(scaler.transform(_))
elapsed = time.time() - start
metrics['latency_ms_no_tta'] = (elapsed/100)*1000
metrics['throughput_fps_no_tta'] = 100/elapsed

# Per-class metrics
p_per_class, r_per_class, f1_per_class, support =
precision_recall_fscore_support(
    y_test_enc, y_pred_enc,
labels=np.arange(len(label_encoder.classes_)), zero_division=0
)

# Detailed per-class analysis
from collections import defaultdict

class_details = defaultdict(lambda: {
    'correct': 0,
```

```
    'total': 0,
    'predictions': [],
    'confidences': []
})

for i, (true_label, pred_label) in enumerate(zip(y_test, y_pred)):
    conf = np.max(y_pred_proba_tta[i])

    class_details[true_label]['total'] += 1
    class_details[true_label]['predictions'].append(pred_label)
    class_details[true_label]['confidences'].append(conf)

    if true_label == pred_label:
        class_details[true_label]['correct'] += 1

# Create comprehensive per-class DataFrame
class_performance = pd.DataFrame({
    'class_id': label_encoder.classes_,
    'precision': p_per_class,
    'recall': r_per_class,
    'f1_score': f1_per_class,
    'support': support,
    'correct': [class_details[c]['correct'] for c in
label_encoder.classes_],
    'accuracy': [class_details[c]['correct'] /
class_details[c]['total']
                 if class_details[c]['total'] > 0 else 0
                 for c in label_encoder.classes_],
    'avg_confidence': [np.mean(class_details[c]['confidences'])
                       if class_details[c]['confidences'] else 0
                       for c in label_encoder.classes_]
})

# Add confusion info
confusion_info = []
for true_class in label_encoder.classes_:
    if class_details[true_class]['predictions']:
        preds = class_details[true_class]['predictions']
        incorrect_preds = [p for p in preds if p != true_class]
        if incorrect_preds:
            from collections import Counter
            most_common = Counter(incorrect_preds).most_common(1)[0]
```

```
            confusion_info.append(f"{most_common[0]}
({most_common[1]}x)")
        else:
            confusion_info.append("None")
    else:
        confusion_info.append("N/A")

class_performance['confused_with'] = confusion_info

print("\n" + "="*60)
print("FINAL METRICS (with TTA)")
print("="*60)
for k, v in metrics.items():
    print(f"{k:30s}: {v:.4f}")

print("\n" + "="*80)
print("PER-CLASS PERFORMANCE (All Classes)")
print("="*80)
print(class_performance.to_string(index=False))

print("\n" + "="*80)
print("TOP 10 BEST PERFORMING CLASSES")
print("="*80)
best_classes = class_performance.sort_values('f1_score',
ascending=False).head(10)
print(best_classes[['class_id', 'accuracy', 'f1_score', 'support',
'avg_confidence']].to_string(index=False))

print("\n" + "="*80)
print("TOP 10 WORST PERFORMING CLASSES (Need Attention!)")
print("="*80)
worst_classes = class_performance.sort_values('f1_score').head(10)
print(worst_classes[['class_id', 'accuracy', 'f1_score', 'support',
'confused_with']].to_string(index=False))

# Save detailed report
class_performance.to_csv('per_class_performance_report.csv',
index=False)
print("\n✅ Detailed per-class report saved to
'per_class_performance_report.csv'")

# Confusion matrix
cm = confusion_matrix(y_test, y_pred)
```

```
plt.figure(figsize=(12, 10))
if len(label_encoder.classes_) > 20:
    sns.heatmap(cm, annot=False, cmap='Blues', cbar=True)
else:
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=label_encoder.classes_,
                yticklabels=label_encoder.classes_)
plt.title('Confusion Matrix (with TTA)')
plt.ylabel('True Label')
plt.xlabel('Predicted Label')
plt.tight_layout()
plt.savefig('confusion_matrix_tta.png', dpi=300, bbox_inches='tight')
plt.show()

# Per-class accuracy bar chart
plt.figure(figsize=(16, 8))
class_performance_sorted = class_performance.sort_values('accuracy',
ascending=False)
colors = ['green' if acc > 0.9 else 'orange' if acc > 0.7 else 'red'
          for acc in class_performance_sorted['accuracy']]
plt.bar(range(len(class_performance_sorted)),
        class_performance_sorted['accuracy'],
        color=colors, alpha=0.7, edgecolor='black')
plt.axhline(y=0.9, color='green', linestyle='--', label='90%
threshold')
plt.axhline(y=0.7, color='orange', linestyle='--', label='70%
threshold')
plt.xlabel('Class ID', fontsize=12)
plt.ylabel('Accuracy', fontsize=12)
plt.title('Per-Class Accuracy (Sorted)', fontsize=14,
fontweight='bold')
plt.xticks(range(len(class_performance_sorted)),
           class_performance_sorted['class_id'],
           rotation=90 if len(class_performance_sorted) > 20 else 45)
plt.legend()
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.savefig('per_class_accuracy_bars.png', dpi=300,
bbox_inches='tight')
plt.show()

# F1-Score vs Support scatter plot
plt.figure(figsize=(12, 8))
```

```
scatter = plt.scatter(class_performance['support'],
                      class_performance['f1_score'],
                      c=class_performance['accuracy'],
                      s=100, alpha=0.6, cmap='RdYlGn',
edgecolors='black')
plt.colorbar(scatter, label='Accuracy')
plt.xlabel('Support (Number of Test Samples)', fontsize=12)
plt.ylabel('F1-Score', fontsize=12)
plt.title('F1-Score vs Support per Class\n(Color = Accuracy)',
fontsize=14, fontweight='bold')
plt.grid(True, alpha=0.3)

# Annotate worst performers
worst_5 = class_performance.nsmallest(5, 'f1_score')
for _, row in worst_5.iterrows():
    plt.annotate(f"Class {row['class_id']}",
                 xy=(row['support'], row['f1_score']),
                 xytext=(10, 10), textcoords='offset points',
                 bbox=dict(boxstyle='round,pad=0.5', fc='yellow',
alpha=0.7),
                 arrowprops=dict(arrowstyle='->',
connectionstyle='arc3,rad=0'))
plt.tight_layout()
plt.savefig('f1_vs_support_scatter.png', dpi=300, bbox_inches='tight')
plt.show()

# Confidence analysis per class
plt.figure(figsize=(14, 6))
plt.subplot(1, 2, 1)
plt.scatter(class_performance['accuracy'],
class_performance['avg_confidence'],
            s=class_performance['support']*2, alpha=0.6,
edgecolors='black')
plt.xlabel('Accuracy', fontsize=12)
plt.ylabel('Average Confidence', fontsize=12)
plt.title('Accuracy vs Confidence\n(Size = Support)', fontsize=12,
fontweight='bold')
plt.grid(True, alpha=0.3)
plt.plot([0, 1], [0, 1], 'r--', label='Perfect Calibration')
plt.legend()

plt.subplot(1, 2, 2)
correct_confs = []
```

```
incorrect_confs = []
for i, (true_label, pred_label) in enumerate(zip(y_test, y_pred)):
    conf = np.max(y_pred_proba_tta[i])
    if true_label == pred_label:
        correct_confs.append(conf)
    else:
        incorrect_confs.append(conf)

plt.hist([correct_confs, incorrect_confs], bins=30, label=['Correct',
'Incorrect'],
         alpha=0.7, edgecolor='black')
plt.xlabel('Confidence', fontsize=12)
plt.ylabel('Count', fontsize=12)
plt.title('Confidence Distribution\n(Correct vs Incorrect)',
fontsize=12, fontweight='bold')
plt.legend()
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.savefig('confidence_analysis.png', dpi=300, bbox_inches='tight')
plt.show()

# Save model components
print("\nSaving models...")
model.save('final_cnn_model.keras')
feature_extractor.save('feature_extractor.keras')
joblib.dump(svm, 'final_svm_model.pkl')
joblib.dump(scaler, 'feature_scaler.pkl')
joblib.dump(label_encoder, 'label_encoder.pkl')
print("Models saved successfully!")
```